

Received 6 April 2026, accepted 16 April 2026, date of publication 21 April 2026, date of current version 29 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3686308

APPLIED RESEARCH

VeriGen: An LLM-Augmented Framework for End-to-End Automation of Software Development Lifecycle–From Requirements Specifications to Code Generation

MUDASSAR ADEEL AHMED^{ID}, MUHAMMAD WASEEM ANWAR^{ID}, WASI HAIDER BUTT^{ID},
AND FAROOQUE AZAM^{ID}

Department of Computer and Software Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology (NUST), Islamabad 44000, Pakistan

Corresponding author: Mudassar Adeel Ahmed (mudassar.ce18ceme@student.nust.edu.pk)

This work was supported in part by the Higher Education Commission of Pakistan via the National Research Program for Universities (NRPU) Model-driven Framework for Simultaneously Supporting Multiple Representations of Embedded System Designs to Improve Productivity (MRED) Initiative under Grant 20-15651.

ABSTRACT In recent years, Large Language Models (LLMs) and Generative Artificial Intelligence (GenAI) have gained significant momentum in the automation of software development. Although, previous research studies have also explored specific sub tasks like requirements formalization, UML class diagram generation, UML sequence diagram generation, and skeletal code generation, these efforts are still fragmented and do not provide an integrated, end-to-end software solution. Furthermore, unreliable outputs are frequently caused by instability, omissions, and hallucinations in LLM-generated artifacts. We propose VeriGen, a novel end-to-end generative AI framework for automated software development, to overcome these limitations. VeriGen begins with requirements written in plain natural language, which are structured using RUPPs templates into Structured Natural Language (SNL). LLM outputs are systematically processed through a unique Verifier-Optimizer loop at each subsequent stage, such as class diagram generation, sequence diagram generation, and skeletal Java code synthesis. The Verifier-Optimizer loop identifies correct, incorrect, missing, and extra instances while refining outputs for stability and correctness. This ensures the accuracy and completeness of generated artifacts in addition to the automation. By reliably generating verified and optimized outputs throughout the software development pipeline, VeriGen has been validated against four benchmark case studies, proving its superiority over baseline LLM generated outputs. The results confirm VeriGen as a significant and major step toward achieving a reliable, generative, end-to-end software engineering automation.

INDEX TERMS Generative artificial intelligence (GenAI), large language models (LLMs), end-to-end software development, structured natural language (SNL), RUPPs templates, UML class diagram generation, sequence diagram generation, code generation, verification and optimization, VeriGen tool.

I. INTRODUCTION

The relentless pursuit of efficiency, reliability, and acceleration in software engineering has long been fueled by

The associate editor coordinating the review of this manuscript and approving it for publication was Maria Chiara Caschera^{ID}.

the vision of automation. From the structured paradigms of Model-Driven Engineering (MDE) to the linguistic capabilities of Natural Language Processing (NLP), the field has continuously sought ways to bridge the gap between human intent and machine-executable code. The recent advent of Generative Artificial Intelligence (GenAI), particularly

Large Language Models (LLMs), has ignited a paradigm shift, presenting an unprecedented opportunity to realize this vision. These models, with their profound ability to understand and generate human language and code, promise a future where software can be architected from a simple narrative description of its requirements. The prospect of an end-to-end, AI-driven software development pipeline from natural language requirements to verified design models and a skeletal code structure represents a potential leap forward in how we conceive, design, and build complex systems.

However, a significant gap exists between this promise and the current reality. While LLMs demonstrate remarkable proficiency in generating coherent text and code snippets, their application to the precise, deterministic, and logically consistent domain of software engineering reveals profound limitations. The core issue lies in the probabilistic nature of LLMs, which makes them prone to hallucinations (generating incorrect or non-existent elements), omissions (failing to capture critical requirements), and inconsistencies (producing conflicting information across different artifacts). These shortcomings are not merely inconveniences; they represent critical failures in a process where correctness is paramount. Relying solely on a raw LLM for software development is risky; it can generate creative but structurally flawed outputs. This philosophy mirrors successful hybrid AI approaches in other domains, such as combining attention based deep learning with swarm optimization for predictive maintenance in Industry 4.0 [40].

The problem is compounded by a fragmented research landscape, where sophisticated automated solutions are developed in isolation. As evidenced by the literature, we have witnessed significant advances in automating individual stages of the software development lifecycle (SDLC) in isolation. For instance, notable research has been conducted on formalizing natural language requirements into Structured or Controlled Natural Languages (CNLs) [1], [2], [3]. Parallel efforts have made strides in the automated generation of UML class diagrams from textual descriptions [4], [5], [7], [9], [10], [11], [12], [13], [14], while another body of work focuses on synthesizing sequence diagrams from various inputs [15], [16], [17], [18], [20], [21]. Furthermore, the generation of code from models or text has been a fertile area of investigation [25], [26], [27], [28], [29], [30].

However, this focus on specialization is itself the problem. It has resulted in isolated tools that do not connect to form a complete pipeline. The output of a requirements formalization tool [1] is not seamlessly fed into a diagram generator [9], whose verified output, in turn, does not automatically become the input for a code synthesis engine [29]. This lack of integration forces developers to act as the “glue” between these disparate tools, manually transferring, interpreting, and, most critically, verifying and correcting the outputs at each stage. This manual intervention reintroduces the very bottlenecks and potential for error that automation seeks to eliminate. Moreover, the critical challenge of systematic

verification and optimization of LLM outputs across this pipeline remains largely unaddressed, with most approaches relying on limited rule-based checks or extensive manual refinement [6], [16], [17].

Therefore, the pressing need is not for another point solution that optimizes a single task, but for a holistic, integrated framework that spans the entire software development pipeline. What is required is a system that not only connects these stages but also embeds a robust mechanism to ensure the correctness, consistency, and completeness of the generated artifacts at every step as shown in the figure 1.

This paper introduces VeriGen, an end-to-end generative AI framework that integrates a dedicated Verifier-Optimizer loop to overcome the reliability gaps of LLMs such as hallucinations, omissions, and inconsistencies and the fragmentation of existing automated software engineering research. The framework transforms natural language requirements into verified Structured Natural Language (SNL), synthesizes verified UML class and sequence diagrams, and finally generates compilable (syntactically correct, ready to compile) skeletal Java code. By embedding continuous verification at each stage, VeriGen repositions LLMs as controlled components within a disciplined engineering workflow, bridging automation valleys and providing a trustworthy, integrated pathway from requirements to code.

To address these dual challenges, VeriGen is proposed as an integrated, end-to-end framework that embeds continuous verification and optimization within a generative AI pipeline. The framework operates through three principal stages, each reinforced by a Verifier–Optimizer loop:

- 1) **Verified Requirements Formalization:** Natural language requirements are transformed into Structured Natural Language (SNL) using an LLM, then verified against the original input to identify correct, incorrect, missing, and extra elements. The optimizer refines the SNL to ensure completeness and correctness.
- 2) **Verified UML Model Synthesis:** The optimized SNL guides the generation of class and sequence diagrams via an LLM. The verifier checks syntactic, semantic, and functional consistency, while the optimizer corrects discrepancies, producing reliable design models.
- 3) **Skeletal Code Transformation:** The verified class diagram is systematically mapped into compilable Java code skeletons, ensuring traceability and structural fidelity from requirements to implementation.

By integrating verification at each stage, VeriGen repositions the LLM from a black-box generator to a controlled component within an engineered workflow. This approach mitigates LLM limitations, bridges research fragmentation, and provides a trustworthy, automated pathway from natural language specifications to verified software artifacts. The primary contributions of this research study are fourfold:

- 1) **A Novel End-to-End Framework:** We present the design and implementation of VeriGen, the first integrated framework to automate the complete software development trajectory from natural language

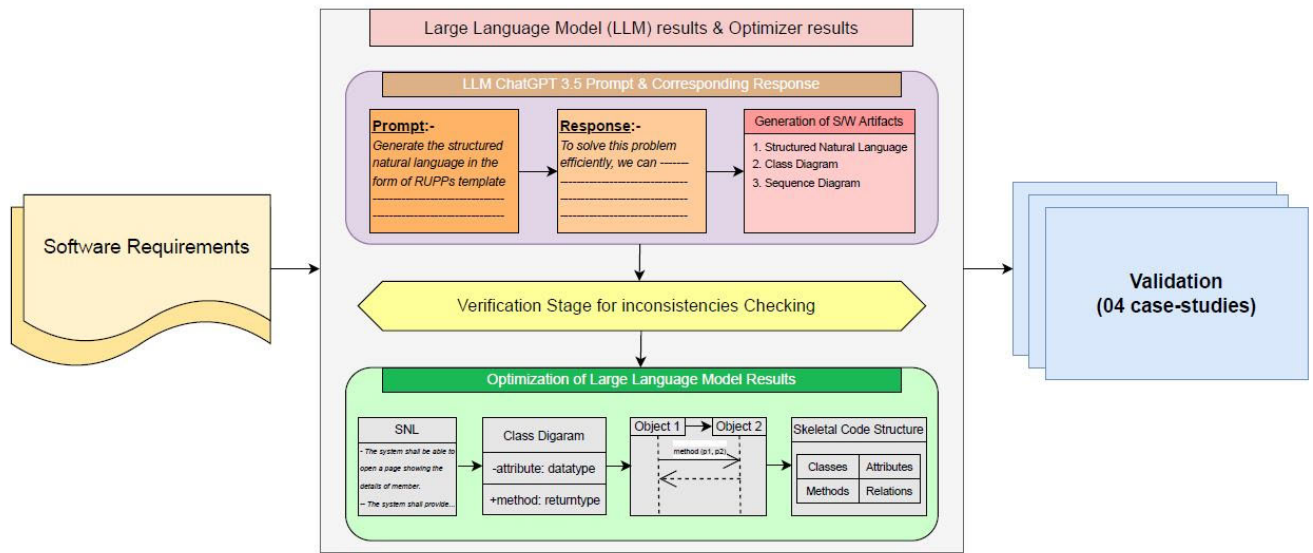


FIGURE 1. Overview of research.

requirements to skeletal code via a continuous pipeline of verified and optimized artifacts.

- 2) The Verifier-Optimizer Methodology: We introduce a novel methodology for coupling generative AI with a dedicated, iterative verification and optimization loop. This mechanism systematically categorizes errors and drives improvement at every stage, ensuring output stability and correctness that far surpasses raw LLM performance.
- 3) A Concrete Tool and Empirical Validation: We describe the VeriGen tool that instantiates this framework and present a comprehensive validation based on 4 case studies. The results provide robust evidence that VeriGen consistently produces more stable, complete, and satisfactory outputs than using an LLM in an isolated, unverified manner [39].
- 4) A Blueprint for Reliable GenAI in SE: By directly addressing the fragmentation and reliability issues that have plagued prior research, VeriGen offers a practical and effective blueprint for how Generative AI can be responsibly integrated into professional software engineering workflows to enhance productivity without compromising on quality. A detailed discussion of how VeriGen addresses the fragmentation and reliability issues compared to prior works is presented in Section II-E (Research Gap).

This paper is structured as follows. Section II surveys related work, highlighting research fragmentation and the identified gap. Section III details VeriGen formal three-phase methodology and its Verifier-Optimizer loop. Section IV describes the tool's implementation, architecture, and modules. Section V presents an end-to-end Library Management System case study with generated artifacts. Section VI provides

a rigorous multi-case evaluation, demonstrating superiority over baseline LLMs. Section VII contains discussion, limitations, and future work. The conclusion section VII summarizes VeriGen's key contributions toward reliable, end-to-end software engineering automation.

II. LITERATURE REVIEW

Automation of software development through Generative AI, Natural Language Processing (NLP), and Model-Driven Engineering (MDE) has been an active research area. However, a careful review of existing studies, sourced from renowned digital libraries including ACM [32], IEEE [33], Springer [34], Elsevier [35], and other reputable journals/conferences from the year 2020-25 (as shown in Table 1) reveals that most contributions are isolated solutions, addressing individual sub-problems such as requirements formalization, UML diagrams generation, and code generation.

TABLE 1. Database-wise Conference/Journal distribution of selected researches.

Sr#	Databases	References	Total
1	Springer: Journal	[1], [25]	2
	Springer: Conference	[27]	1
2	IEEE: Journal	[23], [30]	2
	IEEE: Conference	[6], [7], [9], [28]	4
3	ACM: Conference	[5], [17], [26]	3
4	Elsevier: Journal	[15]	1
	Elsevier: Conference	[4], [8], [10], [19]	4
5	Others (Conf.)	[2], [3], [18]	3
6	Others (Journals)	[11]–[14], [16], [20]–[22], [24], [29]	10
Total			30

Despite promising advances, these works stop short of delivering a holistic, end-to-end system. This limitation

TABLE 2. Summary of key studies on LLM applications in software engineering.

RPs	Input	Tool Used	Tools Developed	Algorithm Used	Output	Validation
SNL Generation						
Veizaga et al. [1]	NL Reqs.	WordNet, VerbNet, NLTK, Xtext	Rimay	N/A	CNL	4 SRSs (460 Reqs.)
Davila et al. [2]	NL Queries	Gemini LLM, Prolog, InsurLE	N/A	N/A	CNL	Case study (Insurance domain)
Rodrigues et al. [3]	BI Reqs.	CNL-BI, ITLingo ASL	N/A	N/A	CNL	Case study (Utility domain)
Class Diagram Generation						
Babaalla et al. [4]	Textual Reqs.	StanfordNLP, SpaCy, BERT, GPT	N/A	Naive Bayes, RNN, LSTM	Class Diagram	Conceptual/Theoretical analysis
Yang et al. [5]	Textual Reqs.	spaCy, coreferee, scikit-learn	Extraction Pipeline	N/A	Class Diagram	Dataset
Wang et al. [6]	Textual Reqs.	ChatGPT, PlantUML	N/A	N/A	UC, Class Diagram	Expert evaluation
Abdelnabi et al. [7]	Survey Paper	N/A	N/A	N/A	N/A	Qualitative analysis of 20 tools
Rosca et al. [8]	UML Class Diagram	Papyrus, Modelio, Visual Paradigm	N/A	N/A	RTE Evaluation	Case study (Hospital Mgmt.)
Abdelnabi et al. [9]	NL Reqs.	Stanford CoreNLP, Heuristics	N/A	N/A	Class Diagram	Case study
Nasiri et al. [10]	User Stories	Stanford CoreNLP, PyEcore, PlantUML	N/A	N/A	Class Diagram	Case study
Nasiri et al. [11]	User Stories	spaCy, SBERT, K-means	N/A	K-means, SBERT	UC, Class Diagram	2 datasets
Rouabhia et al. [12]	UC Description	ChatGPT, PlantUML	Dynamic Enrichment Method	N/A	Class Diagram	23 use cases
Meng et al. [13]	Textual Reqs.	spaCy, scikit-learn	UML Generator	Bernoulli NB, Rule-based	Class Diagrams	130 annotated entries
Ramzan et al. [14]	Textual Reqs.	spaCy, scikit-learn	N/A	Random Forest, SVM, KNN	Class Diagrams	Custom dataset (2,109 samples)
Zhong et al. [15]	NL Reqs.	NLTK, OpenIE, WordNet, PlantUML	Six-step NLP Pipeline	Heuristics, TF-IDF	SysML Diagrams	6 case studies
Akundi et al. [16]	NL Reqs.	spaCy, PlantUML	T2M Framework	Rule-based	Class/SysML	Case studies
Sequence Diagram Generation						
Jahan et al. [17]	User Stories	Stanford CoreNLP, PlantUML, ChatGPT	Rule-based SD Generator	Heuristics, LLM	Sequence Diagrams	Expert eval (100 stories)
Nur et al. [18]	User Stories	NLP, PlantUML	Rule-based SD Generator	Heuristics	Sequence Diagrams	10 scenarios
Mashkoor et al. [19]	Sequence Diagrams	IBM RSA, NFA Tools	Runtime Alignment Checker	Thompson Construction	Validation Result	4 case studies
Alyami et al. [20]	UC Text	Java, PlantUML	Text-based Alignment Tool	Rule-based	Sequence Diagram	ATM case study
Bouziane et al. [21]	Execution Traces	AspectJ, CPN Tools	Reverse Engineering Toolchain	kBehavior Algorithm	Sequence Diagram	Sales application
Alvin et al. [22]	OO Source Code	Soot, Hypergraph Tools	StaticGen	Hypergraph Pebbling	Sequence Diagrams	30 Android apps
Chen et al. [23]	Sequence Diagrams	CCSL, Z3	N/A	SMT, Graph Search	Consistent SD	10 railway cases
Al-Fedaghi [24]	Sequence Diagrams	N/A	Thing Machine	N/A	Behavioral Model	Theoretical reasoning
Ali et al. [25]	NL Reqs., Source Code	Llama-3, Neo4j	RARTG Framework	RAG, Hybrid Indexing	Traceability Links	4 datasets
Code Generation						
Antal et al. [26]	Class Diagram	GPT-4 Vision	AEF	N/A	Java Code	Manual scoring
Sadik et al. [27]	UML+OCL+FIM	GPT-4, PlantUML	AMDD Framework	N/A	JADE/PADE Code	UVF case study
Nair & Thushara [28]	NL Reqs.+UML	spaCy, NLTK, EMF	NL2Code Framework	NLP, MDE	Code Templates	3 case studies
Mahmood et al. [29]	Class/State Models	CodS Tool	Code Swarm	PSO	Java Code	ECS case study
Liu et al. [30]	LeetCode, CWE	ChatGPT, CodeQL	N/A	N/A	Code Snippets	728 problems, 54 CWE

highlights the pressing need for integrated frameworks capable of spanning the entire software development pipeline while ensuring verification, optimization, and stability in outputs. Summary of thirty papers is presented in Table 2.

A. REQUIREMENTS TO STRUCTURED NATURAL LANGUAGE (SNL) GENERATION

Veizaga et al. [1] develop Rimay, a Controlled Natural Language (CNL) and editor, to formalize natural language requirements. Using NLTK, VerbNet, WordNet, and Xtext, they created a domain-adaptable grammar. Input is informal text; the tool enforces structured templates. Validated on four SRSs, it achieved 88% expressiveness, demonstrating effective machine-processable requirement specification but not end-to-end automation.

Davila et al. [2] propose a hybrid architecture combining LLMs and CNLs. The input is a natural language query processed by the Gemini LLM, which parses it for the InsurLE CNL. A Prolog engine then performs logical reasoning. This leverages LLM flexibility for input and CNL precision for deterministic output, mitigating hallucination by using symbolic AI for the final reasoning step.

Rodrigues et al. [3] apply CNLs to specify Business Intelligence applications, introducing CNL-BI and ITLingo ASL. Input is natural language requirements for BI. The methodology, demonstrated with the MEDBuddy-BI case study for the NHS, outputs multidimensional models and dashboard specs. Validation via a survey confirmed the utility of the CNLs for generating precise, machine-processable BI specifications. While the reviewed works effectively formalize natural language requirements into structured representations, they operate in isolation, providing no automated path to subsequent design or code generation. VeriGen addresses this by using the RUPPs based SNL as the verified input for downstream UML modeling, thereby bridging the gap from requirements to design.

B. UML CLASS DIAGRAM GENERATION

Babaalla et al. [4] survey automated UML class diagram generation from natural language. Analyzing NLP, ML, and DL approaches, they find limitations in relationship extraction. They propose a novel three-level deep learning architecture using models like BERT for token classification, coreference resolution, and relationship classification. The goal is complete, accurate diagram generation from text with minimal pre-processing and user input.

Yang et al. [5] present an automated pipeline for UML class diagrams from text. Input is free-flowing specifications. Using a Bernoulli Naive Bayes classifier and spaCy for pattern-based rules, it classifies sentences and generates fragments. Evaluated on a crowdsourced dataset of 649 fragments, results showed low precision (17%) and recall (25%), establishing a valuable baseline and benchmark for future research.

Wang et al. [6] study LLM assistance for novice analysts creating UML diagrams. Input is a case study; students

used an LLM. Output was evaluated for correctness. LLMs excelled with atomic elements (e.g., 91.11% for attributes) but failed on relationships (e.g., 24.44% for associations). Human-refined “Hybrid” diagrams scored highest, showing LLMs are assistants, not replacements, for modeling.

Abdelnabi et al. [7] survey tools for generating UML class diagrams from NL requirements, categorizing them by automation level and technique. The review finds no tool fully automates complete diagram generation from free text, citing universal shortcomings in extracting advanced relationships and needing heavy user intervention. The conclusion highlights the problem’s complexity and the need for NLP-AI hybrid solutions.

Rosca et al. [8] benchmark round-trip engineering in Papyrus, Modelio, and Visual Paradigm using a “Hospital Management” case study. They quantitatively and qualitatively assessed forward/reverse engineering and full round-trip scenarios. While isolated tasks succeeded (80%), round-trip engineering revealed semantic inconsistencies. Visual Paradigm had the best output quality, but all tools need better change reconciliation strategies.

Abdelnabi et al. [9] propose a method for UML class diagram generation from unrestricted text. Input is processed with Stanford CoreNLP. Heuristic rules are applied across five phases for extraction and refinement. Output includes advanced relationships. Validation against the UMGAR tool on a common case study showed their approach extracted a more complete and accurate set of UML elements.

Nasiri et al. [10] automate UML class diagram generation from user stories aligned with MDA. Using Stanford CoreNLP and rule-based extraction on agile user stories (input), they output an XMI model. Compared to Visual Narrator on an online course case study, their method achieved 98% precision, successfully extracting a more complete set of elements, including attributes and operations.

Nasiri et al. [11] develop an AI framework automating backlog refinement and UML generation. Input is user stories; tool uses heuristic rules, k-means clustering, and Sentence-BERT (paraphrase-mpnet-base-v2) to group stories and detect duplicates. The refined backlog is then transformed into UML diagrams. Validation on 102 and 160 stories showed 98% precision in similarity detection, reducing manual effort.

Rouabhia et al. [12] use ChatGPT to dynamically enrich UML class diagrams. Input is natural language use case descriptions for a “Waste Recycling Platform.” The AI tool suggests methods, which are iteratively incorporated into a PlantUML diagram. Validation showed 22 methods added with high behavioral alignment, demonstrating feasibility for Agile diagram enhancement, though it depends on input quality.

Meng et al. [13] propose an NLP framework for UML class diagrams from text. Input is textual requirements. The tool uses a Bernoulli Naive Bayes classifier (TF-IDF features) and spaCy for a four-step pipeline. Trained on 600 diagrams, it achieved 88.46% classification accuracy.

The approach generates diagrams but is limited by input quality and predefined rules.

Ramzan et al. [14] use a hybrid approach for UML diagram generation. Input is text processed with spaCy for element extraction. A machine learning model (Random Forest, SVM, KNN) predicts relationships. Trained on a custom dataset of 2,109 samples, their best model achieved 96% accuracy, improving relationship extraction over purely rule-based methods, though it is limited to four relationship types. Despite significant progress in extracting class diagrams from text, existing approaches either require heavy manual intervention, fail to capture complex relationships, or do not produce verified outputs suitable for direct integration with code generation. VeriGen's Verifier Optimizer loop ensures that the generated class diagrams are structurally complete and semantically aligned with the requirements, providing a reliable foundation for implementation.

C. UML SEQUENCE DIAGRAM GENERATION

Zhong et al. [15] propose an end-to-end method for generating SysML diagrams from unstructured text. Using NLTK, OpenIE, and WordNet, the input is processed via six steps including tf-idf and relationship extraction. Outputs are BDD, IBD, and REQ diagrams. Validated on six corpora, it achieved up to 91.5% precision in phrase extraction, providing a standardized start for systems engineers.

Akundi et al. [16] present a rule-based T2M framework for UML/SysML diagrams from text. The tool uses spaCy for NLP and offers an interactive interface for user refinement. Input is natural language; output is a diagram via PlantUML. Evaluation on a UAV case study showed high recall but lower precision, balancing automation with necessary manual oversight.

Jahan et al. [17] compare rule-based and LLM (ChatGPT) generation of sequence diagrams from user stories. The rule-based tool uses Stanford CoreNLP. Evaluation on 100 stories showed rule-based methods were effective for simple stories (80% accuracy), while ChatGPT produced more complex, context-rich diagrams, sometimes overcomplicating them. LLMs show potential but require careful handling.

Nur et al. [18] create a rule-based system for sequence diagrams from Behat Gherkin scenarios. The input is Given-When-Then text. The tool applies NLP and heuristic rules to generate PlantUML code. Validation against manual diagrams for ten scenarios showed 100% accuracy in object and flow structure but only 40% accuracy in semantically correct object naming.

Mashkoo et al. [19] address live verification of system behavior against UML sequence diagrams. Input is runtime message traces and sequence diagram specs. Their tool, a plugin for IBM RSA, compiles diagrams into NFAs for efficient runtime checking. Validated on four systems, it successfully detected misalignments between specified and actual behavior.

Alyami et al. [20] develop an educational tool for transforming use case diagrams into sequence diagrams. Input is a PlantUML text description of a use case diagram. A custom Java program performs a syntactic mapping to generate PlantUML for a sequence diagram. Demonstrated with an ATM case study, it enhances consistency for students but is limited to core elements.

Bouziane et al. [21] reverse engineer sequence diagrams from execution traces. Input is traces from bytecode instrumentation (AspectJ). Using Colored Petri Nets and the kBehavior algorithm, they filter noise and merge traces to output High-Level Sequence Diagrams (HLSDs) with conditions. Validated on a "Sales" app, it successfully extracted conditional fragments but is limited to single-threaded systems.

Alvin et al. [22] present StaticGen, a static analysis framework for sequence diagrams from source code. Input is code; a Code Hypergraph is built. A query system lets users refine the output. Proven equivalent to Message Sequence Charts, it was validated on 30 Android apps, efficiently helping to narrow down diagrams for tasks like vulnerability discovery.

Chen et al. [23] formally verify consistency among UML sequence diagrams for safety-critical systems. Input is diagrams with combined fragments. The method transforms them into CCSL constraints. Two verification methods (SMT-based and lightweight graph-based) were evaluated on railway cases. The graph method was 36–241 times faster, effectively detecting common inconsistencies like cycles.

Al-Fedaghi et al. [24] critiques UML sequence diagrams and proposes the Thing Machine (TM) model as an alternative. The input is a system interaction description. The TM model uses "thimacs" and five generic actions to create a more precise, less ambiguous representation. Remodeling an ATM withdrawal SD demonstrated clearer separation of static and dynamic aspects. Current methods for sequence diagram generation are often rule based or rely on limited LLM prompts without systematic verification, leading to incomplete or inconsistent behavioral models. VeriGen incorporates a dedicated sequence diagram verifier that checks lifelines, messages, and interaction fragments against the validated SNL, followed by optimizer rules that correct errors, ensuring behavioral consistency with the requirements.

D. CODE GENERATION FROM MODELS AND TEXT

Ali et al. [25] propose RARTG, a traceability link generation tool between NL requirements and code. Input is requirements; the tool uses Llama 3 to enrich code summaries and combines keyword, vector, and knowledge graph indexes. The LLM then predicts traceable classes. Evaluation on four datasets showed improved F1-scores, with a hybrid index strategy performing best.

Antal et al. [26] evaluate GPT-4-Vision for generating Java code from UML class diagram images. Input is diagram images with prompts of varying detail. Output code was

manually scored. It performed well on single-class diagrams but variably on multi-class ones, averaging 88.25% correctness. It sometimes added functionality but also omitted elements.

Sadik et al. [27] propose an AMDD framework using GPT-4 for code generation. Input is UML models (class, activity, state) with OCL constraints and FIPA ontology in PlantUML text. Output is deployment-ready Java/Python code for a multi-agent system. Validation on a UVF case study showed the code matched expected behavior and even introduced logical improvements.

Nair and Thushara [28] present NL2Code, a hybrid NLP-MDE framework. Input is natural language requirements and UML diagrams. The tool uses NLP for text analysis and MDE for transformation into code templates. Evaluated on three case studies, it achieved 93% accuracy, reducing development time by 50% and cost by 40% through automated, high-quality code generation.

Mahmood et al. [29] develop Code Swarm (CodS), a tool for generating Java code from UML models. Input is class/state machine diagrams. Unlike rule-based tools, it uses Particle Swarm Optimization (PSO) to learn transformation rules from examples. Validated on an Elevator Control System, it correctly transformed 98.9% of constructs, suggesting nearest matches when no exact rule existed.

Liu et al. [30] empirically evaluate ChatGPT's code generation. Input is LeetCode problems and CWE scenarios. Output code was assessed for correctness, understandability, and security. Performance was better on pre-2021 problems (68.41% acceptance). Security was a major concern (33.32% of code had vulnerabilities), though providing CWE descriptions in a dialog helped fix 89.4% of issues. Existing code generation tools typically accept only a single input (e.g., class diagrams or natural language) and lack integrated verification, resulting in code that may be structurally flawed or inconsistent with the original requirements. VeriGen generates skeletal Java code directly from the verified class diagram, maintaining traceability from requirements through design to implementation, and produces compilable code that is both structurally faithful and verifiably aligned with the earlier artifacts.

Recent work has further explored complementary aspects of LLM based code generation. Kumar et al. [40] proposed a hybrid deep learning and swarm intelligence approach for predictive maintenance in industrial robotics, demonstrating how attention based models combined with optimization can improve reliability, a philosophy that parallels VeriGen verifier optimizer loop. Dou et al. [41] conducted an extensive empirical study of bugs in LLM generated code, developing a bug taxonomy and a self critique method for iterative repair, which directly informs the verification mechanisms in our framework. Jonnala et al. [42] evaluated the computational efficiency (execution time, memory usage) of Python code generated by GPT models, showing that Chain of Thought prompting significantly improves efficiency without sacrificing correctness, a finding that

supports the design of VeriGen optimizer rules. Finally, Bai et al. [43] introduced a collaborative framework where an intelligent agent generates detailed prompts (role definition, task instructions) to guide LLMs through a multi stage code generation process (demand optimization, code writing, code review), achieving substantial gains and code quality. While these works address different sub problems (bug analysis, efficiency measurement, prompt engineering), they collectively underscore the value of systematic, multi stage verification and optimization, a principle that VeriGen embodies in an end to end pipeline from requirements to code.

E. RESEARCH GAP

Generative AI and Large Language Models (LLMs) hold transformative potential for software engineering by enabling automation from natural language requirements to code. However, their adoption is hindered by significant reliability gaps. LLMs are probabilistic models that excel at generating plausible text but lack rigorous reasoning capabilities, leading to critical failure modes: hallucinations (generating incorrect or unsupported content), omissions (failing to capture explicit or implied requirements), inconsistencies (producing conflicting artifacts), and a verifiability gap (lacking inherent validation mechanisms). These issues make raw LLMs unreliable for mission-critical development, necessitating an architecture that integrates verification alongside generation.

The reviewed literature demonstrates substantial progress in automating specific stages of software development, from requirements formalization to code generation. However, all contributions remain fragmented, solving problems in isolation or at the maximum some research studies like [16] and [22] etc. are dealing with code generation along with class/ sequence diagram generation. Existing tools either require significant human intervention or fail to address verification loop, leading to inconsistent or incomplete outputs. Large Language Models, while promising, frequently suffer from hallucinations, omissions, and poor relationship handling. Critically, no prior work provides an end-to-end generative AI pipeline that integrates requirements, UML diagrams (both structural & behavioral), and code generation while a continuous Verifier-Optimizer loop to ensure systematic verification and optimization.

In contrast to prior fragmented research, VeriGen is designed as an integrated, end-to-end solution. While substantial progress has been made in isolated areas like requirements formalization [1], [2], [3], UML diagram generation [4], [5], [6], [7], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [20], and code synthesis [26], [27], [28], [29], [30], these efforts remain disconnected, creating "automation valleys." VeriGen addresses this by applying a Verifier-Optimizer loop at every stage. Starting with requirements expressed in Structured Natural Language (SNL) using RUPPs templates, it transforms these into verified SNL, synthesizes verified UML class and sequence diagrams, and finally maps the verified class diagram into

compilable skeletal Java code. This approach repositions the LLM from a black-box generator to a controlled component within an engineered workflow, mitigating LLM unreliability, bridging research fragmentation, and ensuring a trustworthy, automated pathway from requirements to code.

III. PROPOSED METHODOLOGY

The proposed tool, VeriGen, establishes a systematic pipeline for the automated transformation of natural language requirements into structured design models and skeletal code structures. The methodology is designed to mitigate the limitations of Large Language Models (LLMs), which, while effective in generating initial artifacts, often produce outputs that are incomplete, redundant, or structurally inconsistent. To address these shortcomings, VeriGen integrates three core components: (i) generation and verification of Structured Natural Language (SNL) in the RUPPs template form, (ii) design generation through class and sequence diagrams, and (iii) skeletal Java code structure generation. Each component involves an LLM for artifact generation, a verifier for consistency checking, and an optimizer for refinement, ensuring a bias-free pipeline that emphasizes correctness, traceability, and structural fidelity as shown in the Figure 2:

A. PHASE I: STRUCTURED NATURAL LANGUAGE GENERATION AND VERIFICATION

The pipeline begins with the transformation of natural language requirements into Structured Natural Language (SNL) expressed in the RUPPs template form. This initial transformation is performed by the LLM, which produces candidate SNL statements. Because LLM generated outputs are prone to error, the SNL Verifier is applied immediately to assess structural and semantic validity. The verifier does not assume correctness but evaluates whether the generated SNL reflects the underlying requirements, categorizing instances into correct, incorrect, missing, and extra instances. Correct instances faithfully capture requirements, incorrect instances contain structural or semantic deviations, missing instances represent untransformed requirements, and extra instances capture unsupported outputs introduced by the LLM.

After verification, the SNL Optimizer applies a set of heuristic rules that enforce compliance with the RUPPs template. These rules restructure conditionals (e.g., if-then, when, while), standardize actor-driven actions into system-provided capabilities, and formalize system-initiated obligations. The result is an improved SNL specification that is syntactically uniform, semantically precise, and traceable to the original requirements. This optimized SNL serves as the foundation for subsequent design model generation.

1) LLM SNL VERIFIER

The first stage of the proposed verification framework is the Structured Natural Language (SNL) Verifier, which is responsible for ensuring that the requirements translated by the Large Language Model (LLM) into SNL conform to the syntactic and semantic conventions prescribed by the RUPPs

template. Since SNL instances generated in this template serve as the foundational input for downstream modeling activities including the derivation of class diagrams, sequence diagrams, and ultimately skeletal code structures. Any inaccuracies at this level are prone to propagate throughout the modeling pipeline, leading to inconsistencies and structural misalignments in subsequent artifacts. Consequently, the role of the verifier is critical in maintaining the fidelity of the requirements to model transformation process.

The verification procedure unfolds in three interrelated phases. In the initial preprocessing phase, both the raw requirements and their corresponding LLM-generated SNL instances are decomposed into atomic units to establish a one-to-one alignment. This segmentation enables systematic mapping of each requirement to its SNL counterpart and provides the basis for fine-grained analysis. The second phase focuses on coverage validation, where the verifier examines whether each requirement in the input set has been faithfully represented in structured form, while simultaneously ensuring that no extraneous or unsupported SNL instances have been introduced. In the final phase, structural correctness is assessed by evaluating each SNL statement against the canonical syntactic templates defined in the RUPPs framework, such as conditional constructs (if-then clauses), actor-oriented affordances, and system-initiated obligations. This step ensures that the SNL not only reflects the intended semantics of the original requirement but also adheres to the standardized patterns necessary for its transformation into formal models as shown in the Figure 3. Importantly, the Verifier is not a Large Language Model; it is a deterministic rule based engine that applies syntactic and semantic heuristics (e.g., POS tagging, keyword matching, template comparison) without invoking any LLM.

The outcome of this verification process is the categorization of SNL instances into four distinct classifications. Correct instances are those that are both syntactically compliant and semantically faithful to their corresponding requirements. Incorrect instances, by contrast, are those where an SNL statement is generated but fails to maintain structural validity or semantic alignment with the source requirement. Missing instances occur when a requirement fails to produce an SNL counterpart, thereby creating a traceability gap. Extra instances arise when the LLM produces statements that cannot be traced back to any requirement, thus introducing redundancy and noise into the model space. Together, these classifications provide a comprehensive diagnostic view of the LLM's performance at this stage and form the foundation for subsequent optimization procedures.

2) OPTIMIZATION RULES FOR STRUCTURED NATURAL LANGUAGE (SNL) GENERATION

Structured Natural Language (SNL) requirements constitute the foundational layer for automated UML model derivation. The transformation of informal, often ambiguous textual requirements into SNL ensures syntactic uniformity,

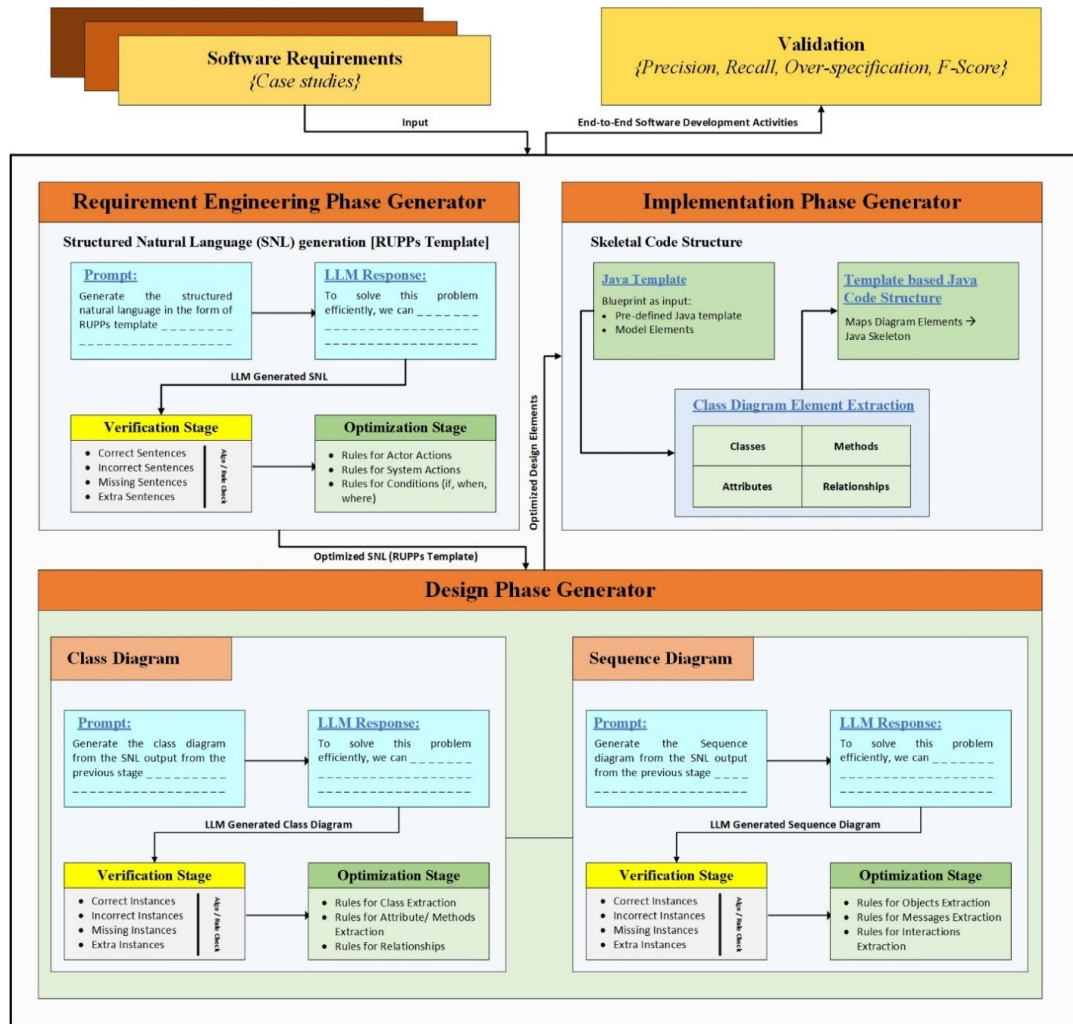


FIGURE 2. End-to-end framework.

semantic clarity, and model traceability. The following heuristics delineate the systematic conversion of raw requirements into validated SNL templates, thereby minimizing interpretative variance and enabling their seamless integration into downstream modeling stages such as class and sequence diagram construction.

1) Conditional Statement Rules

Rule SN-1: If-Condition Transformation

Principle: Requirements formulated as conditional statements with an initiating clause “if” must adhere to the canonical SNL template:

If <condition> then the system shall <action>.

This enforces explicit separation between the antecedent (triggering condition) and the consequent (system obligation).

Library Example:

Requirement: “If the entered information is wrong then system asks member to reenter the details.”

SNL: “If the entered information is wrong then the system shall ask the member to reenter the details.”

Rule SN-2: When-Condition Transformation

Principle: Requirements commencing with “when” must capture temporal triggers in the structured form: When <event> the system shall <action>.

This ensures that all event-driven scenarios are normalized as deterministic system responses, avoiding ambiguities of implicit timing.

Library Example:

Requirement: “When the member logs in, the system authenticate the credentials.”

SNL: “When the member logs in the system shall authenticate the credentials.”

Rule SN-3: While-Condition Transformation

Principle: Statements introduced with “while” must be formalized to represent persistent or iterative system behavior, using the structure:

While <process> the system shall <action>.

This explicitly captures concurrency and ongoing activities that are often overlooked in free-text requirements.

Library Example:

Requirement: “While uploading a file, the system displays a progress bar.”

<pre> FUNCTION verify_snل(لlm_output, requirement_set): correct_instances = [] incorrect_instances = [] # structurally invalid or misaligned SNL missing_instances = [] # requirement not represented in SNL extra_instances = [] # SNL statement not traceable to a requirement # --- Step 1: Preprocess & Align --- requirement_list = split_into_individual_requirements(requirement_set) snل_list = split_into_individual_snل_statements(لlm_output) # --- Step 2: Verify Coverage --- FOR EACH req IN requirement_list: IF find_matching_snل(req, snل_list) IS NONE: missing_instances.append({"requirement": req, "reason": "No SNL generated"}) FOR EACH snل_statement IN snل_list: IF find_matching_requirement(snل_statement, requirement_list) IS NONE: extra_instances.append({"snل": snل_statement, "reason": "No corresponding requirement"}) # --- Step 3: Verify Structural Correctness -- - FOR EACH snل_statement IN snل_list: linked_req = find_matching_requirement(snل_statement, requirement_list) IF linked_req IS NOT NONE: is_valid = validate_snل_structure(snل_statement, linked_req) IF is_valid IS TRUE: correct_instances.append({"requirement": linked_req, "snل": snل_statement}) ELSE: error_reason = diagnose_snل_issue(snل_statement, linked_req) incorrect_instances.append({ "requirement": linked_req, "snل": snل_statement, "reason": error_reason }) RETURN correct_instances, incorrect_instances, missing_instances, extra_instances </pre>	<pre> # --- Helper Function Examples --- FUNCTION validate_snل_structure(snل, req): IF req begins with "If" OR "When" OR "While" OR "Where": IF snل does NOT begin with the same keyword: RETURN FALSE IF "shall" is missing in the consequence clause: RETURN FALSE IF the condition from req is absent: RETURN FALSE ELSE IF subject_of(req) == "system": IF snل does NOT start with "The system shall": RETURN FALSE IF main verb from req is not lemmatized in snل: RETURN FALSE ELSE IF subject_of(req) is an actor (e.g., Member, Librarian, Administrator): IF snل does NOT match "The system shall provide [actor] with the ability to ...": RETURN FALSE RETURN TRUE FUNCTION diagnose_snل_issue(snل, req): IF req is conditional AND "then" is missing in snل: RETURN "Missing 'then' in conditional structure" IF subject_of(req) == "system" AND "shall" is missing: RETURN "Obligation keyword 'shall' missing" IF subject_of(req) is actor AND structure is not ability-based: RETURN "Actor action not transformed into ability-based form" RETURN "General structural mismatch" </pre>
--	---

FIGURE 3. Pseudocode for LLM SNL verifier.

SNL: “While uploading a file the system shall display a progress bar.”

Rule SN-4: Where-Condition Transformation

Principle: Requirements beginning with “where” must encode contextual restrictions or domain constraints in the structured form:

Where <condition> the system shall <action>.

This ensures environmental or situational preconditions are preserved in the requirements model.

Library Example:

Requirement: “Where a user attempts to log in with an incorrect password three times, the system locks the account.”

SNL: “Where a user attempts to log in with an incorrect password three times, the system shall lock the account.”

2) System Action Rules

Rule SN-5: System-Initiated Actions

Principle: Whenever the system itself is the subject of the requirement, the obligation must be framed as:

The system shall be able to <verb phrase>.

This removes passive or vague phrasing and enforces explicit representation of functional responsibilities.

Library Example:

Requirement: “The system displays the log-in page.”

SNL: “The system shall be able to display the log-in page.”

3) Actor Action Rules

Rule SN-6: Actor-Initiated Actions (Present Tense)

Principle: For present-tense requirements describing actor-initiated interactions, the responsibility must be reframed as a system-provided capability, following the template:

The system shall provide <actor> with the ability to <verb phrase>.

This shifts the emphasis from actor autonomy to system-enabled affordances.

Library Example:

Requirement: “The Member clicks the log-in button on the Home Page.”

SNL: “The system shall provide the member with the ability to click the log-in button on the Home Page.”

Rule SN-7: Actor-Initiated Actions (Future Tense)

Principle: For requirements phrased in future tense, the actor’s intended action must still be encapsulated within a system-provided affordance:

The system shall provide <actor> with the ability to <future action>.

This maintains tense-agnostic consistency and ensures predictive or planned interactions are equally modeled.

Library Example:

Requirement: “Members should be logged into the system.”

SNL: “The system shall provide members with the ability to log in.”

4) Other Action Rules

Rule SN-8: Unspecified Subject Actions

Principle: Requirements where no explicit subject is provided (implicit functional needs) must be reformulated as system capabilities:

The system shall be able to <action> <object>.

This guarantees that all functional intentions are grounded in system accountability.

Library Example:

Requirement: “Books should be available to reserve.”

SNL: “The system shall be able to reserve available books.”

5) Actor Identification Rules

Rule SN-9: Actor Extraction

Principle: Actors are systematically identified through linguistic analysis of requirement text, specifically nouns (NN, NNS, NNP, NNPS) denoting external roles or stakeholders. The system itself is excluded to avoid redundancy. Consistent reference resolution must be applied across requirements to ensure coherent actor mapping in subsequent models.

Library Example:

Extracted actors from the Library Management System include Member, Librarian, Administrator, Guest.

B. PHASE II: DESIGN GENERATION THROUGH CLASS AND SEQUENCE DIAGRAMS

In the second phase, the optimized SNL is used as input to generate design-level artifacts. The LLM, guided by structured prompts, produces class diagrams and sequence diagrams, which capture the structural and behavioral perspectives of the system, respectively. Given that these outputs are error prone, VeriGen integrates two specialized verifiers: the Class Diagram Verifier and the Sequence Diagram Verifier.

The Class Diagram Verifier examines the LLM generated diagram by checking whether the classes, attributes, methods, and relationships implied by the SNL are adequately represented. Rather than assuming correctness, the verifier identifies each element as correct, incorrect, missing, or extra. Similarly, the Sequence Diagram Verifier evaluates lifelines, messages, interaction order, and control fragments (alt, opt, loop) against the behavioral constraints derived from requirements. Both verifiers provide a diagnostic assessment, revealing inconsistencies and gaps that need to be addressed.

Following this diagnostic stage, optimizers refine both class and sequence diagrams. For class diagrams, optimization rules consolidate redundant class candidates, extract attributes from descriptive nouns, and lemmatize methods derived from verbs, while also refining relationships based on their contextual semantics. For sequence diagrams, rules derive lifelines from system actors and classes, synthesize messages from actions, identify parameters from noun phrases, and formalize interaction fragments for conditionals and loops. The result is a pair of optimized UML diagrams that are both structurally consistent and semantically aligned with the verified SNL.

1) LLM CLASS DIAGRAM VERIFIER

The second stage of the verification framework is the Class Diagram Verifier, which assesses the structural integrity of the class diagram generated by the LLM when given optimized SNL as input. Since the class diagram constitutes the backbone of object-oriented modeling by defining classes, attributes, methods, and their inter-relationships, inaccuracies at this stage can cascade into subsequent sequence diagrams and skeletal code structures. The role of the verifier is therefore to identify whether the abstraction produced by the LLM aligns with the logical and structural constraints implied by the requirements, and to flag deviations in terms of correct, incorrect, missing, and extra instances.

The verification process is carried out in four neutral phases, each addressing a different structural component as shown in the Figure 4. The first phase examines the classes derived from the SNL requirements and checks whether they are adequately reflected in the LLM generated diagram. This does not assume correctness but instead determines which required classes are present, which are omitted, and whether any additional unsupported classes have been introduced. The second phase evaluates attributes by

verifying whether system properties and identifiers expressed in the requirements are appropriately represented, while also flagging attributes that are misplaced or redundant. The third phase focuses on methods, ensuring that functional operations implied by the requirements are captured in the diagram and identifying cases where they are missing or incorrectly represented. Finally, the fourth phase validates the relationships among classes, analyzing their existence and structural correctness with respect to multiplicity, direction, and semantic role. This phase treats relationships not as expected or given, but as subject to verification for alignment with requirement-driven constraints.

The outcome of this process is a diagnostic classification of all structural elements into four categories. Correct elements are those consistent with the requirements and structurally sound. Incorrect elements exist in the diagram but are misrepresented, such as relationships with erroneous multiplicities or misplaced methods. Missing elements highlight constructs that should have been derived from requirements but are absent in the diagram, while extra elements capture constructs introduced by the LLM that lack traceability to the requirements. Through this categorization, the Class Diagram Verifier offers a bias-free mechanism to evaluate the fidelity of LLM outputs, thereby establishing a reliable basis for subsequent optimization through heuristic rules.

2) OPTIMIZATION RULES FOR CLASS DIAGRAM GENERATION

Class diagrams form the structural backbone of the system model. The following rules optimize the extraction of classes, attributes, methods, and relationships from validated SNL requirements, incorporating the provided core NLP rules.

1) Class & Object Extraction Rules

Rule CD-1: Noun-Based Class Candidate Identification

Rule: All nouns identified by POS tags NN (singular), NNS (plural, to be converted to singular), NNP (proper singular), and NNPS (proper plural) are primary candidates for classes. Redundant candidates must be consolidated into a single unique class.

Example:

From the text, nouns such as *member*, *book*, *librarian*, and *administrator* are identified as initial class candidates. *Books* (NNS) is converted to *Book* (NN).

Rule CD-2: Compound Concept Unification

Rule: A bi-gram of consecutive NN/NNP/NNS tags must be merged into a single class name using a hyphen or CamelCase to form a cohesive concept. An NNS tag in the bi-gram is first converted to its singular form.

Example:

Book ID (NNP NN) is merged into *BookId*.

Rule CD-3: Proper Noun Class Validation

Rule: A word tagged as NNPS (Proper Noun, Plural) should be considered a valid class.

Library Example:

A term such as *The Archives*, tagged as NNPS, forms the class *Archive*.

Rule CD-4: Contextual Filtering for Non-Class Nouns

Rule: A noun (NN/NNS) should be discarded as a class candidate if its immediate linguistic context indicates it is likely an attribute or part of a verb phrase:

- It is part of a noun phrase followed by an adjective (JJ), conjunction (CC), possessive pronoun (PRP\$), or verb (VBZ, VBN).
- It is preceded by a preposition (IN), infinitive marker (TO), or modal verb (MD).
- It is followed by TO or MD, even if it ends a sentence.

Library Example:

- "*required (JJ) information (NN)*": information is filtered out; it is an attribute, not a class.
- "*entered (VBN) information (NN)*": information is filtered as the object of the verb.
- "*details (NNS) include (VBZ)*": details is retained as a valid class (*Detail*) due to domain-specific relevance.
- "*ability to (TO) reserve (VB)*": ability is filtered due to the subsequent TO.

Rule CD-5: End-of-Sentence Noun Validation

Rule: A noun (NN/NNS) immediately followed by a sentence-ending full stop (.) is a strong candidate for a valid class.

Library Example:

"The books should be stored in the Cabinet." validates *Cabinet* as a class.

2) Attribute & Method Extraction Rules

Rule CD-6: Attribute Identification via Modification

Rule: Nouns filtered out by Rule CD-4 are primary candidates for attributes. Additionally, nouns describing properties or identifiers of a class are treated as attributes.

Library Example:

login ID, *password*, *return date*, *total number*, *fine amount*, and *book title* are attributes of the *Member*, *Book*, and *Loan* classes.

Rule CD-7: Method Identification via Verb Association

Rule: Verbs (VB, VBD, VBG, VBN, VBP, VBZ) associated with a class define its methods. The lemma of the verb forms the method name.

Library Example:

- *clicks* → *click()*
- *displays* → *display()*
- *validates* → *validate()*
- *checks* → *check()*

Rule CD-8: Lemmatization of Operations

Rule: All verbs extracted for methods must be

```

FUNCTION verify_class_diagram(llm_diagram,
requirement_elements):

    correct_elements = []
    incorrect_elements = [] # e.g., wrong
multiplicity, misplaced attributes
    missing_elements = [] # elements
derived from requirements but absent in
diagram
    extra_elements = [] # elements
present in diagram but not traceable to
requirements

    # --- Step 1: Verify Classes ---

    FOR EACH element IN
requirement_elements.classes:
        IF element IN llm_diagram.classes:

correct_elements.append({"type": "Class",
"name": element})
        ELSE:

missing_elements.append({"type": "Class",
"name": element})

        FOR EACH element IN
llm_diagram.classes:
            IF element NOT IN
requirement_elements.classes:
                extra_elements.append({"type":
"Class", "name": element})

    # --- Step 2: Verify Attributes ---

    FOR EACH attr IN
requirement_elements.attributes:
        IF attr IN llm_diagram.attributes:

correct_elements.append({"type":
"Attribute", "name": attr})
        ELSE:

missing_elements.append({"type":
"Attribute", "name": attr})

        FOR EACH attr IN
llm_diagram.attributes:
            IF attr NOT IN
requirement_elements.attributes:
                extra_elements.append({"type":
"Attribute", "name": attr})

    # --- Step 3: Verify Methods ---

    FOR EACH method IN
requirement_elements.methods:
        IF method IN llm_diagram.methods:
            correct_elements.append({"type":
"Method", "name": method})
        ELSE:
            missing_elements.append({"type":
"Method", "name": method})

    FOR EACH method IN llm_diagram.methods:
        IF method NOT IN
requirement_elements.methods:
            extra_elements.append({"type":
"Method", "name": method})

    # --- Step 4: Verify Relationships ---

    FOR EACH rel IN
requirement_elements.relationships:
        IF rel IN llm_diagram.relationships:

            # Check structural details like multiplicity,
direction, or type

            IF
rel.is_equivalent(llm_diagram.relationships[rel])
:
                correct_elements.append({"type":
"Relationship", "name": rel})
            ELSE:

incorrect_elements.append({"type":
"Relationship", "name": rel, "reason": "Incorrect
details"})
        ELSE:
            missing_elements.append({"type":
"Relationship", "name": rel})

    FOR EACH rel IN llm_diagram.relationships:
        IF rel NOT IN
requirement_elements.relationships:
            extra_elements.append({"type":
"Relationship", "name": rel})

    RETURN correct_elements, incorrect_elements,
missing_elements, extra_elements

```

FIGURE 4. Pseudocode for LLM class diagram verifier.

converted to their base form (lemma) to ensure grammatical consistency.

Library Example:

validates → *validate()*, *displays* → *display()*,
checks → *check()*.

3) Relationship Identification Rules

Rule CD-9: Relationship Discovery from Verbal Phrases

Rule: A verb connecting two recognized classes indicates a relationship. Ownership phrases imply aggregation or composition, while transitive actions imply association.

Library Example:

Member reserves Book → Association between *Member* and *Book*, often realized through an intermediary class such as *Reservation*.

3) LLM SEQUENCE DIAGRAM VERIFIER

The third stage of the verification framework is the Sequence Diagram Verifier, which evaluates the behavioral interactions

produced by the LLM when transforming optimized SNL requirements into sequence diagrams. Sequence diagrams are critical for capturing dynamic system behavior, as they model message exchanges among objects over time. Inaccuracies in these diagrams, such as misplaced messages, missing interactions, or incorrect ordering, can compromise the interpretability of system workflows and lead to flawed code generation in later stages. The purpose of the verifier is therefore not to assume the LLM output as valid but to neutrally assess whether the generated sequence diagram reflects requirement-driven behavioral constraints, and to identify discrepancies in terms of correct, incorrect, missing, and extra instances.

The verification process proceeds through several phases of analysis as shown in the Figure 5. The first phase validates participating objects, ensuring that the lifelines instantiated in the diagram correspond to the actors and system components implied by the requirements, while also identifying any omissions or unsupported additions. The second phase focuses on messages, verifying that the

method calls and interactions expressed in the requirements are present, while flagging messages that are either absent or extraneous. The third phase examines interaction order, ensuring that the sequence of messages respects the logical flow of events described in the requirements rather than being misplaced or reordered arbitrarily by the LLM. The final phase extends this verification to interaction fragments, including conditions, optional flows, and iterative loops. This step ensures that control structures such as if-then conditions or optional user choices are accurately captured and that no invalid or redundant fragments are introduced.

The results of the verifier are categorized into four diagnostic outcomes. Correct elements represent objects, messages, and interaction fragments that align with the requirements and exhibit structural validity. Incorrect elements capture cases where generated interactions exist but are misrepresented, such as messages placed in the wrong order or conditions lacking proper guard expressions. Missing elements refer to objects or interactions described in the requirements but not reflected in the diagram, while extra elements denote those introduced by the LLM without any basis in the requirements. This diagnostic classification provides a systematic evaluation of behavioral consistency at the interaction level and establishes a neutral baseline for subsequent optimization, where heuristic rules refine and improve the LLM-generated sequence diagrams.

4) OPTIMIZATION RULES FOR SEQUENCE DIAGRAM GENERATION

Sequence diagrams model the dynamic interactions between objects instantiated from the class diagram. The construction is directly driven by the flow of events described in the SNL requirements. Object detection is carried out from the same rules as stated in the A1. Other significant extractions are stated as below.

1) Lifeline & Message Extraction Rules

Rule SD-1: Lifeline Derivation from Interacting Instances

Rule: For a given use case scenario, the participating objects are instances of the classes involved. The initiating actor and key system components become the primary lifelines.

Library Example:

For the “Reserve Book” scenario: librarian: Librarian, reservationPage: ReservationPage, reservationController: ReservationController, bookDB: BookDatabase.

Rule SD-2: Message Synthesis from SNL Actions

Rule: Every verb in an SNL action statement translates to a synchronous message between lifelines. The message signature is derived from the method lemmas identified in Rule CD-7 and CD-8.

Library Example:

SNL: “The system shall validate the entered information.”

Becomes: reservationController -> bookDB: validate-BookDetails(bookId)

Rule SD-3: Parameterization from Sentence Objects

Rule: Nouns following a verb in an SNL statement become the parameters for the corresponding message or method call.

Library Example:

SNL: “...enter his login Id with password.”

Becomes: member -> loginPage: enterCredentials(loginId, password)

Rule SD-4: Return Message Implication

Rule: Every synchronous message (solid arrow) implies a corresponding return message (dashed arrow) carrying the result of the operation, even if not explicitly stated.

Library Example:

The message reservationController -> bookDB: check-Availability(bookId) is followed by bookDB -> reservationController: isAvailable.

2) Interaction Fragment Rules

Rule SD-5: alt Fragment Mapping for Conditional Logic

Rule: Conditional SNL statements (“If [condition] then ...”) are mapped to alt combined fragments. The condition becomes the guard expression.

Library Example:

```
alt
[book is available]
reservationController -> bookDB: reserve(bookId,
memberId)
bookDB -> reservationController: success
else
[book is not available]
reservationController -> reservationPage: displayError-
errorMessage(“Not Available”)
end
```

Rule SD-6: opt Fragment Mapping for Optional Steps

Rule: Optional actions described in requirements (e.g., “may choose to”, “can ignore”) are modeled using an opt fragment.

Library Example:

```
opt [user chooses to print]
user -> system: requestPrint()
system -> printer: printDetails(details)
end
```

Rule SD-7: loop Fragment Identification for Multiplicity

Rule: Actions performed on plural nouns or described using quantifiers such as “each”, “all”, or “for every” must be enclosed in a loop fragment.

Library Example: loop [for each book in category]
system -> bookDB: getBookDetails(bookId)

<pre> FUNCTION verify_sequence_diagram(llm_sequence, requirement_elements): correct_elements = [] incorrect_elements = [] # e.g., misplaced messages, wrong sender/receiver missing_elements = [] # interactions described in requirements but absent in diagram extra_elements = [] # interactions present in diagram but not traceable to requirements # --- Step 1: Verify Objects --- FOR EACH obj IN requirement_elements.objects: IF obj IN llm_sequence.objects: correct_elements.append({"type": "Object", "name": obj}) ELSE: missing_elements.append({"type": "Object", "name": obj}) FOR EACH obj IN llm_sequence.objects: IF obj NOT IN requirement_elements.objects: extra_elements.append({"type": "Object", "name": obj}) # --- Step 2: Verify Messages / Method Calls --- FOR EACH msg IN requirement_elements.messages: IF msg IN llm_sequence.messages: correct_elements.append({"type": "Message", "name": msg}) ELSE: missing_elements.append({"type": "Message", "name": msg}) FOR EACH msg IN llm_sequence.messages: IF msg NOT IN requirement_elements.messages: extra_elements.append({"type": "Message", "name": msg}) </pre>	<pre> # --- Step 3: Verify Interaction Order --- FOR i FROM 0 TO length(requirement_elements.sequence) - 1: IF i < length(llm_sequence.sequence): IF llm_sequence.sequence[i] == requirement_elements.sequence[i]: CONTINUE ELSE: incorrect_elements.append({ "type": "Sequence", "position": i, "reason": "Order mismatch or misplaced interaction" }) ELSE: missing_elements.append({ "type": "Sequence", "position": i, "reason": "Interaction not present in diagram" }) # --- Step 4: Verify Conditional / Alternative Flows --- FOR EACH cond IN requirement_elements.conditions: IF cond IN llm_sequence.conditions: correct_elements.append({"type": "Condition", "name": cond}) ELSE: missing_elements.append({"type": "Condition", "name": cond}) FOR EACH cond IN llm_sequence.conditions: IF cond NOT IN requirement_elements.conditions: extra_elements.append({"type": "Condition", "name": cond}) RETURN correct_elements, incorrect_elements, extra_elements, missing_elements </pre>
--	---

FIGURE 5. Pseudocode for LLM sequence diagram verifier.

bookDB → system: details
end

Rule SD-8: Actor–System Boundary Delineation

Rule: Messages must clearly distinguish between actor-initiated UI interactions and internal system process messages, separating user intent from implementation logic.

Library Example:

Actor Interaction: member → loginPage: clickLogin-Button()

System Interaction: loginPage → authController: authenticate(credentials)

Rule SD-9: Object Creation for “Adds/Creates” Verbs

Rule: Verbs like “adds”, “creates”, “issues” imply the creation of a new object instance, modeled with a create message.

Library Example:

“The librarian adds a book.”

Becomes: librarian → bookController: addNew-Book(bookDetails)

bookController → newBook: create(bookDetails)

Rule SD-10: Focus of Control Activation

Rule: The execution period of a lifeline after receiving a message must be marked with an activation bar (focus of control), lasting until a return message is sent.

Library Example:

Upon receiving `authenticate(credentials)`, the `authController` lifeline remains active while executing authentication logic before returning `authenticationResult`.

C. PHASE III: SKELETAL CODE STRUCTURE GENERATION

The final phase of the VeriGen pipeline is the generation of skeletal Java code structures from the optimized class diagram. This stage bridges the gap between high-level models and implementation by automatically producing compilable Java files that preserve the structural essence of the system design. The generated skeletal code is explicitly not intended to represent a complete system; rather, it provides a compilable, traceable architectural baseline that developers can extend with domain specific logic. In the current implementation, code generation is driven solely by the verified class diagram; the sequence diagram is used for

verification and consistency checking but is not automatically translated into code.

For each class in the diagram, a Java source file is generated that includes package declarations, imports, class headers, and fields corresponding to attributes. Constructors are synthesized in both default and parameterized forms, and accessor/mutator methods (getters and setters) are generated for all attributes to maintain encapsulation. Method stubs corresponding to operations are also created, complete with signatures, parameter lists, and placeholder bodies that developers can later extend with domain-specific logic. Relationships among classes are preserved by encoding multiplicities: one-to-many associations are represented using List structures, while one-to-one associations are captured through direct object references.

The generated skeletal code is not intended to represent a complete system but provides a structurally faithful, compliant foundation. It maintains strict traceability to the class diagram, ensuring alignment between requirements, design, and implementation.

1) JAVA CODE GENERATION FROM CLASS DIAGRAM ELEMENTS

The final stage of the framework is the automated generation of a skeletal code structure, which transforms the class diagram into a set of Java class definitions. This stage bridges the gap between high-level design models and implementation artifacts by producing compliant yet minimal code structures that capture the architectural essence of the system. The skeletal code does not implement business logic but instead provides a well structured foundation upon which detailed implementation can be built, ensuring that the transition from design to code is both traceable and consistent.

The code generation process follows a modularized pipeline as shown in Figure 6. For each class identified in the class diagram, a Java source file is generated containing the structural and behavioral elements derived from the diagram. Each file begins with a class header, including package declarations and necessary imports, ensuring consistency across the generated system. Attributes defined in the class diagram are translated into private fields with appropriate Java data types, supported by type mapping mechanisms to preserve semantic accuracy. Constructors are automatically synthesized, including a default constructor and parameterized constructors where attributes exist, providing initialization support consistent with object-oriented principles. Accessor and mutator methods (getters and setters) are generated for each attribute, establishing encapsulation and enabling controlled data access.

Beyond basic class definitions, the framework incorporates business methods corresponding to operations specified in the class diagram. These methods are synthesized with correct signatures, including parameters and return types, while their bodies contain placeholder logic to indicate that further implementation is required. This ensures that functional

behavior is consistently scaffolded without prematurely fixing business logic. Relationships among classes, such as associations and aggregations, are represented through appropriate field declarations. Multiplicity is respected by mapping one-to-many associations into Java List structures and one-to-one associations into single object references, thereby preserving design semantics in code.

The outcome of this process is a complete set of skeletal Java files, one per class, containing headers, attributes, constructors, accessors, mutators, method stubs, and relationship representations. While these files do not constitute a fully functional system, they provide a structurally faithful and compliant foundation, derived directly from verified and optimized UML artifacts. This ensures that the generated code is aligned with requirement-driven design, maintains structural integrity, and offers a traceable path from textual requirements to executable software.

Together, these three phases define VeriGen as a robust, bias-free framework for requirements-driven software modeling. By combining LLM-based generation with systematic verification and optimization, VeriGen ensures that artifacts are not accepted uncritically but are rigorously examined and refined. The use of verifiers highlights inconsistencies in terms of correct, incorrect, missing, and extra instances, while optimizers apply heuristic rules to improve structural and semantic accuracy. The pipeline produces standardized and traceable artifacts across all levels from SNL to UML models to skeletal code, thus reducing error propagation, enhancing reliability, and bridging the gap between natural language requirements and executable software designs.

IV. IMPLEMENTATION OF PROPOSED TOOL VeriGen

This section elaborates on the concrete instantiation of the VeriGen framework as a functional, Web-based application tool, developed as a comprehensive system comprising over 5000 lines of code. The implementation, architected with a Python (FastAPI) backend and a TypeScript (ReactJS) front-end, realizes the conceptual architecture described in Section IV, providing an integrated environment for the end-to-end, verified automation of the software development lifecycle. The tool's design prioritizes modularity, scalability, and user accessibility, ensuring that each stage of the pipeline from requirements to skeletal code generation is both robust and transparent. VeriGen System Architecture and Technology Stack is summarized in the Figure 7.

A. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

VeriGen is architected as a Web-based application, fostering a clear separation of concerns between the user interface and the core processing logic. This decoupled design facilitates maintenance and allows for the independent evolution of front-end and back-end components. The high-level system architecture, illustrated in Figure 7, comprises the following technological foundations:

```

FUNCTION generateJavaCode(classDiagram):
    javaFiles = NEW Map<String, String>()

    FOR EACH class IN classDiagram.classes:
        classCode = generateClassHeader(class)
        classCode +=
generateAttributes(class.attributes)
        classCode +=
generateConstructors(class)
        classCode +=
generateAccessMethods(class.attributes)
        classCode +=
generateBusinessMethods(class.methods)
        classCode +=
handleRelationships(class.relationships)
        classCode += "\n\n"
        javaFiles.put(class.name, classCode)

    RETURN javaFiles

FUNCTION generateClassHeader(class):
    RETURN "package com.library.model;\n\n" +
"import java.util.*;\nimport
java.time.*;\n\n" +
"public class " + class.name + "
{\n\n"

FUNCTION generateAttributes(attributes):
    code = ""
    FOR EACH attr IN attributes:
        code += "    private " +
mapType(attr.type) + " " + attr.name + ";\n"
    RETURN code + "\n"

FUNCTION generateConstructors(class):
    code = "    public " + class.name + " ()
{\n\n"
    IF class.attributes.length > 0:
        params = JOIN(" ", FOR EACH attr IN
class.attributes: mapType(attr.type) + " " +
attr.name)
        code += "    public " + class.name +
"(" + params + ") {\n"
        FOR EACH attr IN class.attributes:
            code += "        this." + attr.name
+ " = " + attr.name + ";\n"
        code += "    }\n\n"
    RETURN code

FUNCTION generateAccessMethods(attributes):
    code = ""
    FOR EACH attr IN attributes:
        capName = capitalizeFirst(attr.name)
        code += "    public " +
mapType(attr.type) + " get" + capName + " ()
{\n        return this." + attr.name + ";\n
}\n\n"
        code += "    public void set" +
capName + "(" + mapType(attr.type) + " " +
attr.name + ") {\n        this." + attr.name
+ " = " + attr.name + ";\n    }\n\n"
    RETURN code

FUNCTION generateBusinessMethods(methods):
    code = ""
    FOR EACH method IN methods:
        params = JOIN(" ", FOR EACH param IN
method.parameters: mapType(param.type) + " "
+ param.name)
        code += "    public " +
mapType(method.returnType) + " " +
method.name + "(" + params + ") {\n        //
TODO: Implement logic\n        return " +
getDefaultValue(method.returnType) + ";\n
}\n\n"
    RETURN code

FUNCTION handleRelationships(relationships):
    code = ""
    FOR EACH rel IN relationships:
        IF rel.multiplicity == "many":
            code += "        private List<" +
rel.targetClass + "> " +
rel.targetClass.toLowerCase() + "s = new
ArrayList<>();\n\n"
        ELSE:
            code += "        private " +
rel.targetClass + " " +
rel.targetClass.toLowerCase() + ";\n\n"
    RETURN code

// Helper functions: mapType(),
capitalizeFirst(), getDefaultValue()
// implement type mapping and default value
logic as needed

```

FIGURE 6. Pseudocode for java code generation.

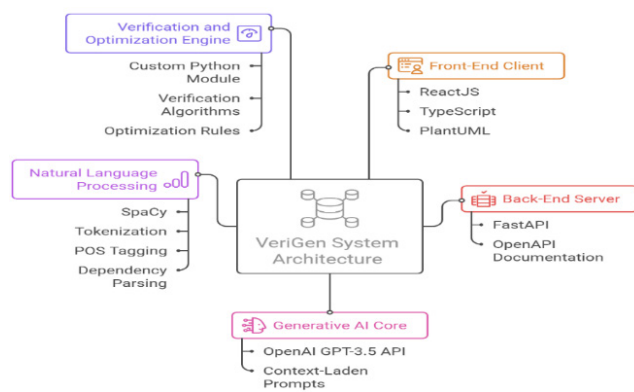


FIGURE 7. VeriGen system architecture.

1) FRONT-END CLIENT

The user interface is implemented using ReactJS with TypeScript. This combination was selected to build a dynamic, type-safe, and responsive single-page application (SPA). It provides users with an interactive canvas to input requirements, visualize the progressive generation

and verification of artifacts (SNL, UML diagrams), and inspect the final skeletal code. For diagram visualization, the front-end integrates the PlantUML library, which renders UML diagrams directly from their textual definitions generated by the back-end.

2) BACK-END SERVER

The server-side logic is built with FastAPI, a high-performance Python web framework. FastAPI was chosen for its asynchronous capabilities, automatic generation of OpenAPI documentation, and rapid development cycle. The back-end serves as the orchestration engine for the entire VeriGen pipeline, managing the sequential execution of each stage and the corresponding Verifier-Optimizer loops.

3) GENERATIVE AI CORE

The primary generative capability is powered by the OpenAI GPT-3.5 API, which acts as the Large Language Model (LLM) for all stages: initial SNL generation, and subsequent class and sequence diagram synthesis. The back-end

constructs sophisticated, context-laden prompts to guide the LLM towards the desired output formats.

4) NATURAL LANGUAGE PROCESSING

The SpaCy library is employed for foundational linguistic processing. It performs essential tasks such as tokenization, part-of-speech (POS) tagging, and dependency parsing on the natural language requirements. These analyses provide critical features that support the verification algorithms and optimization heuristics, particularly in the SNL generation phase.

5) VERIFICATION AND OPTIMIZATION ENGINE

A custom Python module implements the verification algorithms and optimization rules detailed in Section IV. This engine is responsible for parsing LLM outputs, executing the fine-grained comparative analysis against source artifacts (requirements or SNL), and applying the heuristic rules to refine and correct the generated outputs.

B. OPERATIONAL PIPELINE AND CORE MODULES

The operational workflow of VeriGen directly implements its three phase methodology, embedding a dedicated Verifier-Optimizer loop at each stage to ensure reliability. In Module I, natural language requirements are transformed by an LLM into preliminary Structured Natural Language (SNL) using the RUPPs template. The SNL Verifier categorizes each instance as correct, incorrect, missing, or extra, after which the Optimizer applies heuristic rules (e.g., SN-6) to produce a verified SNL specification. This output feeds Module II, where an LLM generates initial UML class and sequence diagrams. These are verified for alignment with the SNL and optimized using rules like CD-1 and SD-5. Finally, Module III performs a deterministic, rule-based mapping from the verified class diagram into compilable skeletal Java code as shown in section V-H, completing the automated pipeline from requirements to implementation foundations.

C. TOOL INTERFACE AND OPERATIONAL WORKFLOW

The VeriGen tool presents a streamlined user experience through its interface. A user's interaction follows a linear, guided workflow as shown in the following Figure 8:

1) Requirement Input

The user pastes the natural language software requirements (e.g., the entire Library Management System description) into a designated text area.

2) Pipeline Execution

The user initiates the automated pipeline. The interface provides visual feedback, progressing through each stage (SNL Generation → Verification → Optimization → UML Synthesis → Code Generation). Crucially, it displays the verification reports, offering transparency by listing the Correct, Incorrect, Missing, and Extra instances identified at each step.

3) Artifact Inspection and Download

Upon completion, the user can review the final, optimized artifacts: the SNL specification, the visualized UML diagrams, and the generated skeletal Java code. The system provides an option to download the complete Java project as a ZIP archive, ready for further development in a standard IDE.

The implementation of the VeriGen tool successfully materializes the proposed framework's theoretical contributions [39]. By leveraging a modern, scalable technology stack and a rigorously engineered modular pipeline, it demonstrates the practical feasibility of end-to-end generative software engineering. The tight integration of the Verifier-Optimizer loops at every stage, empirically validated through the four case studies effectively mitigates the inherent unreliability of raw LLMs. This results in a system that does not just automate software development tasks but does so with a demonstrable commitment to the accuracy, consistency, and stability of its outputs.

V. LIBRARY MANAGEMENT SYSTEM (LMS) CASE STUDY

The Library Management System (LMS) case study, originally outlined by Tiwari et al. [31], is employed in this work to empirically validate the VeriGen framework. By processing the same problem statement, we demonstrate the end-to-end automation capability of our tool. This section presents the complete set of artifacts generated by VeriGen from the initial Structured Natural Language (SNL) to the final skeletal code thereby providing a concrete illustration of the pipeline's output and serving as a basis for evaluating its performance.

A. THE PROBLEM STATEMENT

Library Management System (LMS) problem statement as presented by Tiwari et al. [31] is shown in Figure 9.

B. LLM-GENERATED STRUCTURED NATURAL LANGUAGE (SNL) OUTPUT

The initial transformation of the natural language problem statement into Structured Natural Language (SNL) is performed by the LLM, utilizing the RUPPs template to impose a preliminary structure. As depicted in the Figure 10, the LLM successfully identifies key system actors, actions, and objects from the narrative description, generating a list of candidate SNL statements. This raw output demonstrates the LLM's capability to parse complex requirements and extract salient functional points. However, the generated SNL exhibits characteristic limitations of an unverified, single-pass generation: statements vary in syntactic conformity to the RUPPs template, some requirements are paraphrased rather than formally structured, and implicit relationships between entities are not consistently captured. This output serves as the foundational, yet unrefined, input for the subsequent verification and optimization cycle.

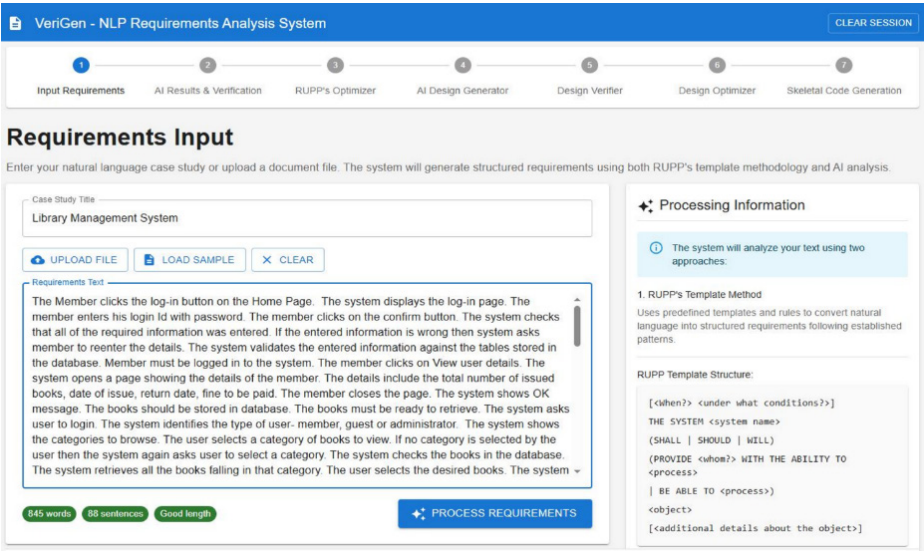


FIGURE 8. Interface of proposed tool VeriGen.

The Member clicks the log-in button on the Home Page. The system displays the log-in page. The member enters his login Id with password. The member clicks on the confirm button. The system checks that all of the required information was entered. If the entered information is wrong then system asks member to reenter the details. The system validates the entered information against the tables stored in the database. Member must be logged in to the system. The member clicks on View user details. The system opens a page showing the details of the member. The details include the total number of issued books, date of issue, return date, fine to be paid. The member closes the page. The system shows OK message. The books should be stored in database. The books must be ready to retrieve. The system asks user to login. The system identifies the type of user- member, guest or administrator. The system shows the categories to browse. The user selects a category of books to view. If no category is selected by the user then the system again asks user to select a category. The system checks the books in the database. The system retrieves all the books falling in that category. The user selects the desired books. The system shows the details of the selected books. The system asks user to print the details. If User does not want to print the details then user can ignore the step. User can reserve a book by inputting the relevant details & the librarian can also reserve a book for a member. User should be logged into the system. User should have correct book Id. Books should be available to reserve. The system asks user to login. The system identifies the type of user- member, guest or administrator. The system shows the categories to browse. The user selects book to reserve. If no book is selected by the user then the system again asks user to select a book to reserve. The user enters Book Id to reserve. If the book Id is wrong then system asks user to recheck the book id. The system checks the books in the database. If the selected book is already reserved on another Id then system asks user to select another book. Members should be available to search. Book should be available to search. The system shows the categories to browse. The member selects a category of searching a book. If no category is selected by the member then the system again asks user to select a category. The system checks the books in the database. The system retrieves all the books falling in that category. Member should give the member Id to the librarian. Books should be available to issue. The librarian Checks the availability of the books. The librarian also checks total number of books issued on that Id. Librarian issues the book. The librarian cannot issue the books if the user has three books issued on his Id. The system updates the information in database. Librarian should be logged into the system. Member should have borrowed books. Member should give the member Id to the librarian. The member returns the book. The librarian enters book id , member id in the system. If the entered book id is incorrect then system asks to re-enter the book id. The system prompts a message that the book with book id is successfully returned. Members should be stored in the database. Members should be available to retrieve by the system. Member must be logged in to the system. Guest can also view members. The user clicks on View members. The system opens a page showing the details of the member. The details include name of member, the total number of issued books, date of issue, return date, fine to be paid. The member closes the page. Librarian should be logged into the system. Books should be available to remove. Book details should be available to add or remove books in the database. The librarian has option of adding or removing a book in database. The system asks librarian to add or remove the book. The librarian adds a book. The system asks librarian to enter all the required details about the new book to be added. The librarian enters the details. If the librarian selects to remove a book then the book should be outdated. The system administrator that all the information has been correctly provided. Administrator should be logged into the system. Member should be available to remove. Member details should be available to add or remove member in the database. The system asks administrator to add or remove a member. The administrator selects to add a member. The system asks administrator to enter all the required details about the new member to be added. The administrator enters the details. If the administrator selects to remove a member then a valid reason of removal is required. The system validates that all the information has been correctly provided.

FIGURE 9. Tiwari et al. [31] library management system problem statement.

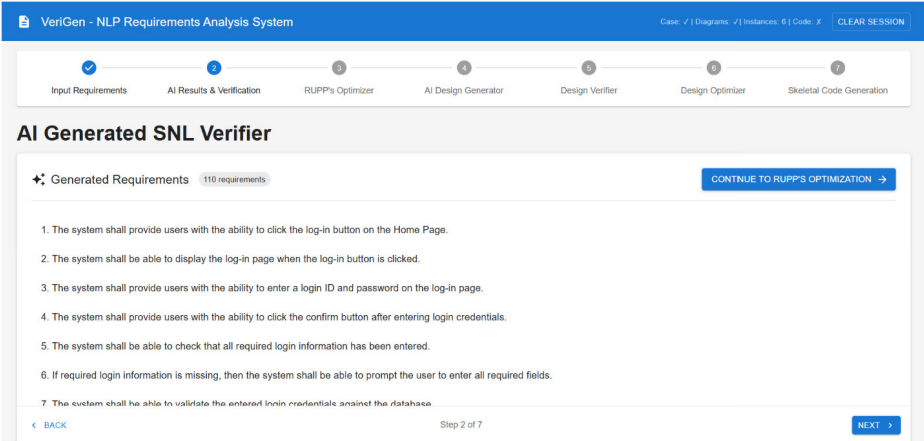


FIGURE 10. LLM generated structured natural language.

C. SNL VERIFICATION (ANALYSIS OF SNL ARTIFACT)

The LLM-generated SNL is systematically analyzed by the SNL Verifier module. The verification results, illustrated in

the corresponding figure, provide a diagnostic breakdown categorizing each instance as Correct, Incorrect, Missing, or Extra relative to the original requirements. Correct

instances are those where the LLM accurately captured and correctly formatted a requirement. Incorrect instances include statements with structural flaws, such as missing obligatory keywords (“shall”) or mis formatted conditional clauses. Missing instances reveal requirements from the original text that the LLM failed to translate into any SNL statement, indicating omissions. Extra instances are SNL statements generated by the LLM that lack a corresponding basis in the input requirements, representing hallucinations. This granular analysis, as shown in Figure 11, quantifies the unreliability of the raw LLM output and precisely identifies the targets for corrective optimization.

D. VeriGen-OPTIMIZED SNL SPECIFICATION

Leveraging the verification report from the verifier, the SNL Optimizer applies a set of heuristic rules (section III-A2) to refine the initial output with the optimization rate of 88% as evident in Table 3. The resulting optimized SNL specification, shown in the provided Figure 12, presents a coherent, standardized, and complete set of requirements. All statements now rigorously adhere to the RUPPs template, with consistent use of conditional constructs (“If...then”), explicit system obligations (“The system shall...”), and properly framed actor affordances (“The system shall provide [Actor] with the ability to...”). Missing requirements have been synthesized and integrated, while incorrect and extra instances have been corrected or removed. This optimized SNL specification is unambiguous, machine-processable, and forms a stable, verified foundation for the subsequent design modeling phase.

E. LLM-GENERATED DESIGN MODELS (CLASS AND SEQUENCE DIAGRAMS)

Using the optimized SNL as input, the LLM is prompted to generate structural and behavioral design models. The accompanying Figure 13 presents the initial UML Class Diagram and Sequence Diagram produced by the LLM. The class diagram identifies candidate classes, attributes, and some relationships, while the sequence diagram outlines key interaction flows for major use cases, such as book reservation. These artifacts demonstrate the LLM’s generative capability in transitioning from textual specifications to visual models. However, the diagrams exhibit typical shortcomings: inconsistencies between the two views (e.g., methods in sequence diagrams not present in the class diagram), incomplete relationship multiplicities, missing key classes or attributes inferred from the SNL, and sequence diagram fragments that lack proper formalization with alt or opt combined fragments for conditional logic.

F. DESIGN VERIFICATION (ANALYSIS OF CLASS AND SEQUENCE DIAGRAM ARTIFACTS)

The Class Diagram and Sequence Diagram Verifiers autonomously analyze the LLM-generated models against the constraints and entities defined in the verified SNL specification. The verification output, displayed in the Figure 14,

provides a detailed audit for each model. For the class diagram, it lists correctly identified elements and flags issues such as missing associations, incorrect multiplicities, and omitted attributes. For the sequence diagram, it verifies the alignment of lifelines and messages with the SNL, identifying missing interactions, incorrectly ordered messages, and the absence of necessary conditional fragments. This stage transforms subjective model assessment into an objective, instance-level classification, clearly describing the gaps and errors that must be addressed to achieve a consistent and complete design.

G. VeriGen-OPTIMIZED UML MODELS (CLASS AND SEQUENCE DIAGRAMS)

Guided by the verification reports, the respective diagram optimizers refine the initial LLM outputs with the optimization rate of 87.8% and 89.7% respectively as evident in Table 8 and Table 9. The final, optimized UML models are presented in the accompanying Figure 15. The class diagram now exhibits a complete set of coherent classes with semantically appropriate attributes, fully specified relationships (including association names and correct multiplicities), and proper application of generalizations. The sequence diagram is structurally formalized, with explicit alt fragments for conditional flows, clearly defined lifelines corresponding to class diagram entities, and a logically accurate sequence of messages that fully captures the system’s dynamic behavior as specified in the SNL. These optimized models are internally consistent and mutually aligned, representing a verified and stable design ready for implementation.

H. SKELETAL JAVA CODE GENERATION FROM VERIFIED CLASS DIAGRAM

In the final transformation stage, the verified and optimized class diagram is automatically mapped to a skeletal Java code structure. The generated code, illustrated in the provided Figure 16, includes a complete set of Java class files. Each class contains private member variables derived from UML attributes, appropriate constructors, getter and setter methods for encapsulation, and method stubs corresponding to the operations identified in the design. Relationships from the class diagram are accurately translated, with associations implemented as object references. The resulting code skeleton is syntactically correct, compilable, and provides a faithful, traceable implementation foundation that directly reflects the validated design, enabling developers to commence detailed programming with a robust and correct architectural baseline.

The performance of the proposed VeriGen tool is evaluated using previously unseen natural language software requirements. Performance assessment is crucial in software development for both consumers and developers. The evaluation of the proposed system incorporates three complementary methodologies informed by established evaluation concepts [36]: adequacy evaluation, which assesses the

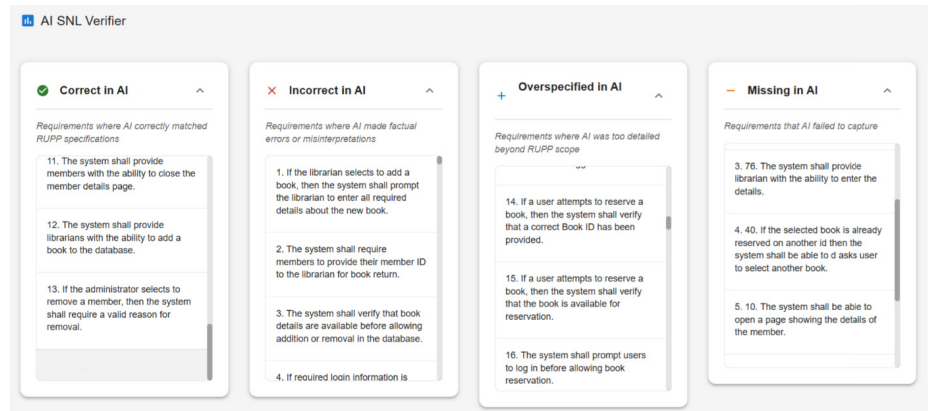


FIGURE 11. LLM SNL results verification.

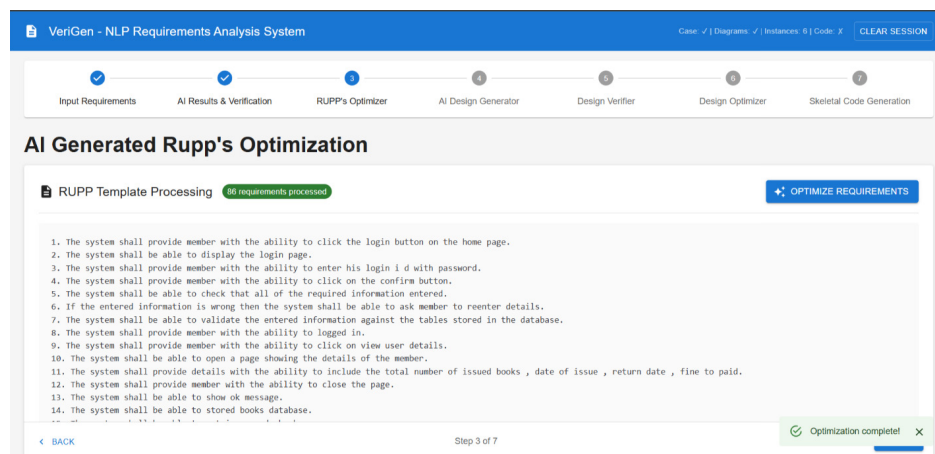


FIGURE 12. VeriGen optimized SNL generation.

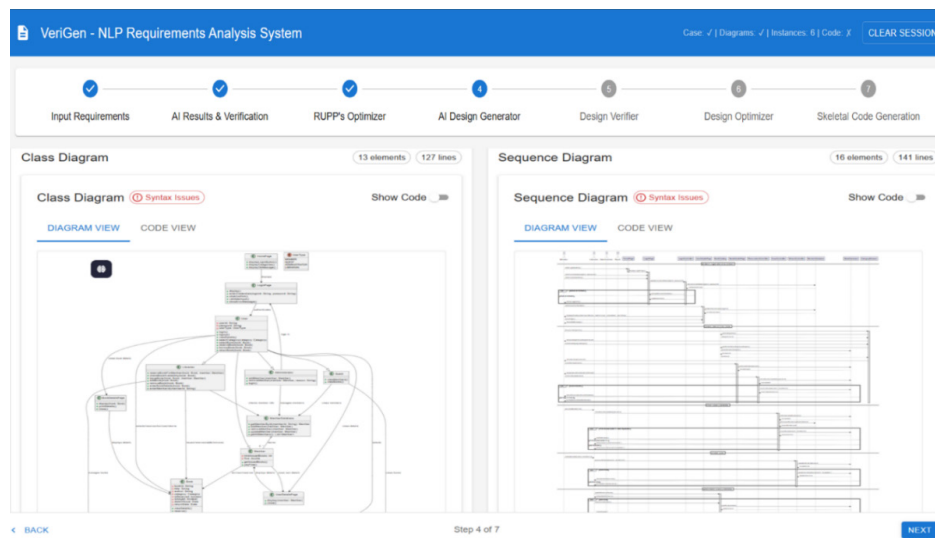


FIGURE 13. LLM generated Class and sequence diagram.

system's overall fitness for its intended task; diagnostics evaluation, used during development to identify flaws

and ensure coverage; and performance evaluation, which quantifies system effectiveness in specific areas. Given the

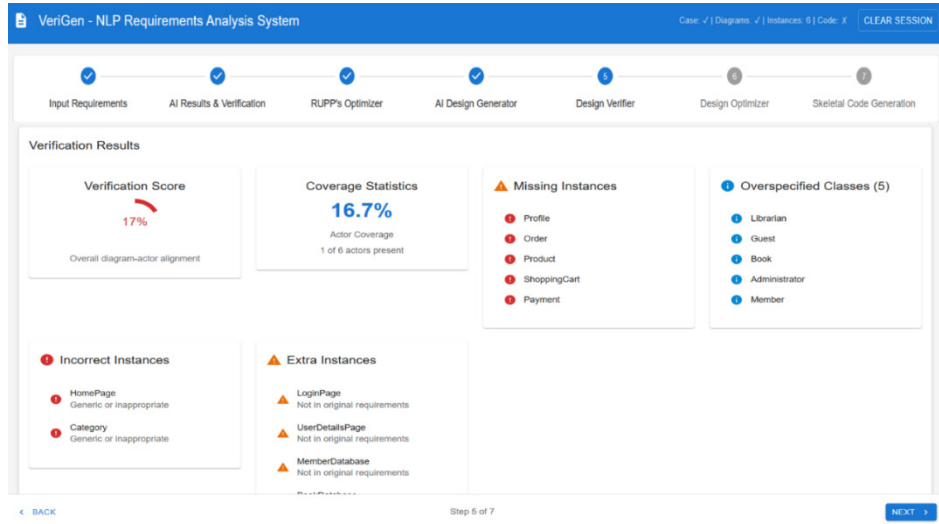


FIGURE 14. LLM design models results verification.

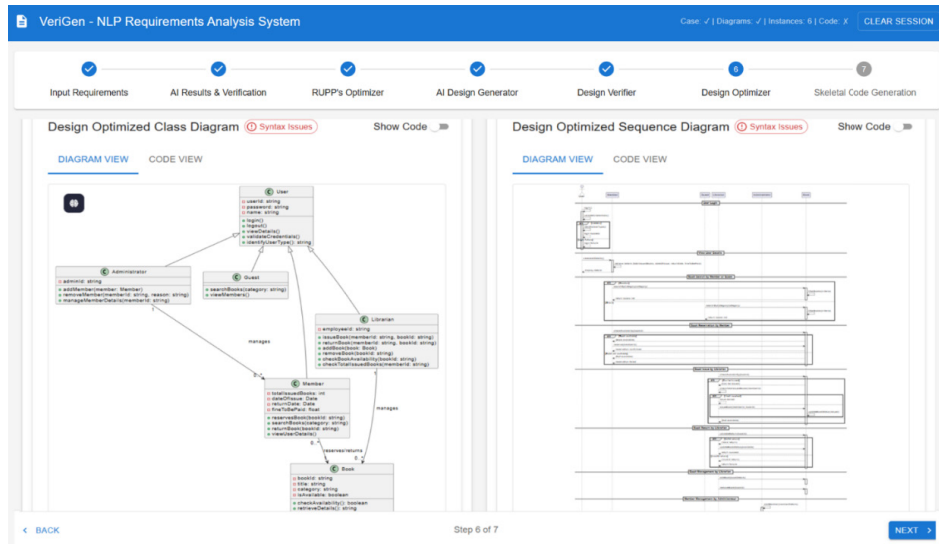


FIGURE 15. VeriGen Optimized class and sequence diagram.

absence of standard benchmarks for evaluating automatically generated software models, domain expert artifacts are used as the correct answer key for Structured Natural Language, UML diagrams, and skeletal code. System performance is measured using standard information retrieval metrics, Recall and Precision, to calculate hit and miss ratios, aiming for values as close to 100% as possible to indicate optimal performance. Precision indicates how much of the extracted knowledge is correct. It is calculated as:

$$\text{Precision} = \frac{N_{\text{correct}}}{N_{\text{correct}} + N_{\text{incorrect}}} \quad (1)$$

N_{correct} is again the number of correct instances, and $N_{\text{incorrect}}$ is the number of incorrect responses from the system. In information extraction, Recall measures the completeness

of the results [37]. It compares correct information produced by the system to the answer key. Recall is defined as:

$$\text{Recall} = \frac{N_{\text{correct}}}{N_{\text{correct}} + N_{\text{missing}}} \quad (2)$$

Here, N_{correct} is the number of correct instances generated by the system, and N_{missing} is the number of elements missed by the system but identified by domain experts.

System performance is evaluated manually by comparing VeriGen generated artifacts such as SNL, UML models, and Java code against a domain expert answer key. A match, either exact or semantically identical, is correct. A non-matching element is incorrect. An element generated but absent from the key is extra, while any key element not produced is

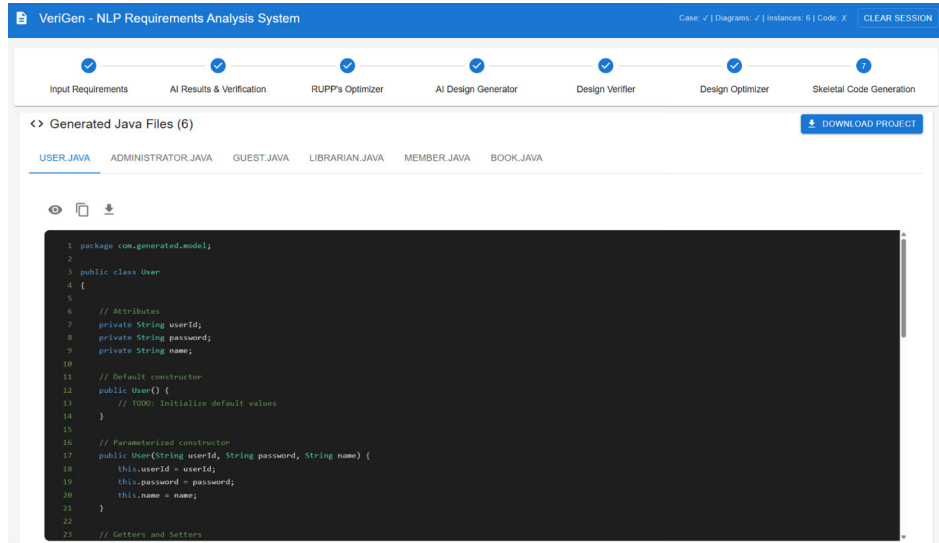


FIGURE 16. Skeletal java code structure.

missing. This method ensures precise assessment of output fidelity against the original problem statement.

Over-specification is another metric, representing extra information not in the answer key. The object-oriented community suggests that over-specification is preferable to missing key information in early analysis stages [38]. It is defined as:

$$\text{Over-specification} = \frac{N_{\text{extra}}}{N_{\text{correct}} + N_{\text{missing}}} \quad (3)$$

N_{extra} is the number of extra elements retrieved, while N_{correct} and N_{missing} are as defined previously.

I. COMPARATIVE ANALYSIS AND PERFORMANCE VALIDATION

To empirically validate the performance and accuracy of the proposed VeriGen framework, a comprehensive comparative analysis was conducted. The evaluation assesses the system's ability to mitigate LLM hallucinations, omissions, and inconsistencies through its integrated Verifier-Optimizer loop across the entire software development pipeline

1) PERFORMANCE MEASUREMENT OF VeriGen ON CASE STUDIES

The VeriGen framework was validated using four distinct case studies from different domains: a Library Management System (LMS), a Monitoring Operating System (MOS), a Zoom Car Booking system (ZCB), and a Digital Home System (DHS). These benchmark case studies [31], comprising natural language requirements of intermediate length and complexity, were not used during the system's development or tuning. The performance was measured at each major artifact generation stage Structured Natural Language (SNL), UML Class Diagram, UML Sequence Diagram, and Skeletal Java Code using the standard metrics of Precision (P), Recall

TABLE 3. SNL generation evaluation summary.

Sr.#	Case study	C	I	M	E	P%	R%	OS%	F%
1	LMS (GPT)	20	17	10	40	54.0	66.6	133	59.6
	LMS (VeriGen)	71	9	7	0	88.7	91.0	0	89.8
2	MOS (GPT)	3	15	4	8	16.6	42.8	114	23.9
	MOS (VeriGen)	20	10	0	2	66.6	100	10	79.9
3	ZCB (GPT)	5	11	13	26	31.2	27.7	144	29.3
	ZCB (VeriGen)	49	4	2	5	92.4	96.0	9.8	94.1
4	DH (GPT)	5	10	7	1	33.3	41.6	8.3	36.9
	DHS (VeriGen)	19	3	2	1	86.3	90.4	4.7	88.3
LLM (ChatGPT) Avg.						33.7	44.6	99.8	37.4
VeriGen Avg.						83.5	94.3	6.1	88.0

Legend: where C, I, M and E indicate the number of Correct, Incorrect, Missing, and Extra Sentences

(R), and Over-specification (OS), as defined in Section VI. Whereas, F-Score (F) is measured by $2 * [(\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})]$.

1) Structured Natural Language (SNL) Generation Performance

The initial and critical phase of the VeriGen pipeline is the transformation of natural language requirements into verified SNL using the RUPPs template. Table 3 presents the detailed evaluation of SNL generation across the four case studies, comparing the outputs of VeriGen against a raw LLM (ChatGPT).

The results demonstrate that VeriGen Verifier-Optimizer loop significantly enhances the reliability of the initial LLM output. VeriGen achieved an average Precision of 83.5% and Recall of 94.3%, with 6.1% Over-specification across all cases. In contrast, the raw LLM showed substantially lower Precision

TABLE 4. Class extraction evaluation summary.

Sr.#	Case study	C	I	M	E	P%	R%	OS%	F%
1	LMS (GPT)	6	2	0	5	75	100	83.3	85.7
	LMS (VeriGen)	6	0	0	0	100	100	0	100
2	MOS (GPT)	4	2	4	1	66.6	50	12.5	57.1
	MOS (VeriGen)	7	0	1	1	100	87.5	12.5	93.3
3	ZCB (GPT)	4	2	2	6	66.6	66.6	100	66.6
	ZCB (VeriGen)	5	1	1	1	83.3	83.3	16.6	83.3
4	DH (GPT)	8	5	0	3	61.5	100	37.5	76.1
	DHS (VeriGen)	8	0	0	0	100	100	0	100
LLM (ChatGPT) Avg.						67.4	79.1	58.3	71.3
VeriGen Avg.						95.8	92.7	7.27	94.1

Legend: where C, I, M and E indicate the number of Correct, Incorrect, Missing, and Extra Classes

(33.7%) and Recall (44.6%), coupled with a very high Over-specification rate (99.8%), indicating prolific hallucination of unsupported statements.

LMS: User(C1), Administrator(C2), Guest(C3), Librarian(C4), Member(C5), Book(C6) **MOS:** Operator(C1), RemoteSensor(C2), MonitoringSystem(C3), Alarm(C4), HelpFacility(C5), Notification(C6), MonitoringLocation(C7), Sensor(C8). **ZCB:** User(C1), Customer(C2), Admin(C3), PaymentSystem(C4), Car(C5) **DHS:** User(C1), Humidistat(C2), Thermostat(C3), Appliance(C5), Alarm(C4), Sensor(C6), PowerSwitch(C7), Planner(C8)

2) UML Class Diagram Generation Performance

The verified SNL from the previous stage served as input for automated UML Class Diagram synthesis. Table 4 summarizes the performance in extracting correct classes. Table 5 and Table 6 provide a granular evaluation of Attributes and Methods per class, respectively. Figure 17 details the accuracy in identifying relationships between classes.

VeriGen achieved perfect (94.1%) scores in extracting classes and (95.4%) scores in extracting attributes across all case studies. Method extraction was also highly robust, with an average Precision of 91.9% and recall of 94.8%. Relationship identification, a known weakness for LLMs, was substantially improved by VeriGen rule-based optimizers, correctly inferring inheritance and association links that were consistently missed or misrepresented by the raw LLM.

3) Overall, Class Diagram and Sequence Diagram Accuracy

To compute an overall score for design model generation, a weighted aggregation was applied to the class diagram components (Classes: 50%, Attributes: 20%, Methods: 20%, Relationships: 10%). The Sequence Diagram accuracy was calculated based on the correct mapping of lifelines between objects and messages (methods), summarized in Table 7 and Table 8.

TABLE 5. Attribute extraction evaluation summary.

Sr.#	Case study	C	I	M	E	P%	R%	OS%	F%
1	LMS	C1:3, C2:1, C3:0, C4:1, C5:4, C6:4	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	100	100	0	100
	MOS	C1:4, C2:2, C3:3, C4:4, C5:1, C6:1, C7:3	C1:0, C2:1, C3:0, C4:0, C5:0, C6:1, C7:0	C1:0, C2:0, C3:1, C4:1, C5:0, C6:0, C7:1	C1:0, C2:0, C3:1, C4:0, C5:1, C6:0, C7:0	90	85.7	9.5	87.7
3	ZCB	C1:3, C2:3, C3:1, C4:3, C5:6	C1:0, C2:0, C3:0, C4:0, C5:1	C1:0, C2:0, C3:0, C4:0, C5:0	C1:0, C2:0, C3:0, C4:0, C5:1	94.1	94.1	5.8	94.1
	DHS	C1:2, C2:3, C3:3, C4:2, C5:2, C6:3, C7:2, C8:1	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0, C7:0, C8:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0, C7:0, C8:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0, C7:0, C8:0	100	100	0	100
LLM (ChatGPT) Avg.						67.1	46.8	14.32	52.6
VeriGen Avg.						96.0	94.9	3.8	95.4

Legend: where C, I, M and E indicate the number of Correct, Incorrect, Missing, and Extra Attributes

TABLE 6. Method extraction evaluation summary.

Sr.#	Case study	C	I	M	E	P%	R%	OS%	F%
1	LMS	C1:5, C2:3, C3:2, C4:6, C5:4, C6:7	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0	100	100	0	100
	MOS	C1:4, C2:2, C3:4, C4:2, C5:3, C6:3, C7:2	C1:0, C2:0, C3:1, C4:0, C5:0, C6:0, C7:0	C1:0, C2:0, C3:1, C4:0, C5:0, C6:0, C7:0	C1:0, C2:0, C3:0, C4:2, C5:0, C6:1, C7:0	95.2	95.2	14.2	95.2
3	ZCB	C1:2, C2:4, C3:4, C4:2, C5:1	C1:0, C2:0, C3:1, C4:0, C5:2	C1:0, C2:0, C3:0, C4:0, C5:1	C1:1, C2:0, C3:0, C4:0, C5:2	81.2	92.8	21.4	86.6
	DHS	C1:5, C2:2, C3:2, C4:2, C5:2, C6:2, C7:3, C8:3	C1:0, C2:1, C3:1, C4:0, C5:0, C6:0, C7:0, C8:0	C1:0, C2:1, C3:1, C4:0, C5:0, C6:0, C7:0, C8:0	C1:0, C2:0, C3:0, C4:0, C5:0, C6:0, C7:0, C8:0	91.3	91.3	0	91.3
LLM (ChatGPT) Avg.						87.4	43.2	52.7	56.4
VeriGen Avg.						91.9	94.8	8.9	93.2

Legend: where C, I, M and E indicate the number of Correct, Incorrect, Missing, and Extra methods

VeriGen achieved an average class diagram accuracy of 87.8% and sequence diagram accuracy of 89.7%,

Library Management System			Corresponding Classes Relationships													
Tool	Rel. Type	C1-C2	C1-C3	C1-C4	C1-C5	C2-C5	C5-C6	C4-C6								
GPT	Inheritance	☒	☒	☒	☒											
	Association					☒	✓	✓								
VeriGen	Inheritance	✓	✓	✓	✓											
	Association					✓	✓	✓								
Monitoring Operating System			Corresponding Classes Relationships													
Tool	Rel. Type	C2-C3	C3-C1	C3-C4	C3-C5	C3-C6	C3-C8	C4-C7	C1-C7	C7-C8						
GPT	Association	☒	☒		☒	☒		☒	☒	☒						
	Composition			☒			☒									
VeriGen	Association	✓	✓		✓	✓		✓	☒							
	Composition			☒			☒			☒						
Zoom Car Booking System			Corresponding Classes Relationships													
Tool	Rel. Type	C1-C2	C1-C3	C2-C4	C3-C6	C4-C5	C4-C6									
GPT	Inheritance	☒	☒													
	Association			✓	✓	☒	✓									
VeriGen	Inheritance	✓	✓													
	Association			☒	✓	☒	☒									
Digital Home System		Corresponding Classes Relationships														
Tool	Rel. Type	C1-C2	C1-C3	C1-C4	C1-C5	C1-C6	C1-C8	C1-C7	C2-C6	C2-C8	C3-C6	C3-C8	C4-C6	C5-C7	C6-C8	C7-C8
GPT	Association	☒	☒	☒	☒	☒	✓	☒	✓	☒	✓	☒	☒	✓	☒	☒
VeriGen	Association	✓	✓	✓	✓	☒	☒	☒	✓	✓	✓	✓	✓	✓	✓	✓

FIGURE 17. Class diagram relationship evaluation.

far surpassing the raw LLM's averages of 56.2% and 56.6%, respectively. The minor discrepancies in the MOS and ZCB case studies are primarily due to the omission of one or two complex methods or relationships in the expert crafted "answer key" model, which were not strictly derivable from the textual requirements but were included based on domain knowledge.

4) End-to-End Pipeline Performance

Finally, the overall performance of the integrated VeriGen pipeline was evaluated by aggregating the weighted scores from each major artifact (SNL: 50%, Class Diagram: 40%, Sequence Diagram: 5%, Code: 5%). The skeletal code accuracy is derived directly from the verified class diagram. The comparative results against the raw LLM are presented in Table 9.

The substantial improvements achieved by VeriGen can be attributed to its systematic Verifier Optimizer loop, which operates at every stage of the pipeline. For Structured Natural

Language generation, the SNL Verifier classifies statements into correct, incorrect, missing, and extra instances; the Optimizer then applies rules such as SN 6 (Actor Initiated Actions) and SN 8 (Unspecified Subject Actions) to reformulate vague or passive requirements into explicit system obligations, ensuring completeness and uniformity. Similarly, for class diagram synthesis, the Class Diagram Verifier identifies missing attributes and relationships, while rules like CD 9 (Relationship Discovery from Verbal Phrases) correctly infer associations that raw LLMs often miss. For sequence diagrams, rules such as SD 5 (alt Fragment Mapping for Conditional Logic) and SD 7 (loop Fragment Identification) transform natural language conditionals and repetitions into properly formalized interaction fragments, improving behavioral accuracy. The cumulative effect of these targeted corrections across all stages is a stable, traceable pipeline that reduces error propagation. The implications are significant: VeriGen demonstrates that generative AI can be harnessed for reliable, end to end software engineering,

TABLE 7. Overall class diagram accuracy.

Case Study	Classes	Attributes	Methods	Relationships	Total
LMS (GPT)	8/8×50=50	7/13×20=10.7	11/27×20=8.1	2/7×10=2.8	71.6
LMS (VeriGen)	8/8×50=50	13/13×20=20	27/27×20=20	7/7×10=10	100
MOS (GPT)	4/8×50=25	5/26×20=3.8	4/25×20=3.2	0/9×10=0	32
MOS (VeriGen)	7/8×50=43.7	18/26×20=13.8	20/25×20=16	5/9×10=5.5	79
ZCB (GPT)	4/6×50=33.3	10/22×20=9	7/17×20=8.2	3/6×10=5	55.5
ZCB (VeriGen)	5/6×50=41.6	16/22×20=14.5	13/17×20=15.2	3/6×10=5	76.3
DH (GPT)	8/8×50=50	5/18×20=5.5	9/23×20=7.8	4/15×10=2.6	65.9
DH (VeriGen)	8/8×50=50	18/18×20=20	21/23×20=18.2	12/15×10=8	96.2
VeriGen (Overall)					87.8
LLM (ChatGPT) Overall					56.2

TABLE 8. Overall sequence diagram accuracy.

Case Study	Objects / Classes	Methods	Total
LMS (GPT)	8/8×50=50	11/27×50=20.3	70.3
LMS (VeriGen)	8/8×50=50	27/27×50=50	100
MOS (GPT)	4/8×50=25	4/25×50=8	33
MOS (VeriGen)	7/8×50=43.7	20/25×50=40	83.7
ZCB (GPT)	4/6×50=33.3	7/17×50=20.5	53.8
ZCB (VeriGen)	5/6×50=41.6	13/17×50=38.2	79.8
DH (GPT)	8/8×50=50	9/23×50=19.5	69.5
DH (VeriGen)	8/8×50=50	21/23×50=45.6	95.6
VeriGen (Overall)			89.7
LLM (ChatGPT) Overall			56.6

reducing the need for manual verification and enabling trustworthy automation in domains where correctness is critical.

VeriGen achieved an overall end-to-end accuracy of 88% across the four case studies, demonstrating a strong, consistent, and reliable automation pipeline. In stark contrast, the use of a raw LLM (ChatGPT) without the Verifier-Optimizer loop yielded a significantly lower and highly unstable accuracy of 46.8%. This 41.2 percentage point improvement underscores the fundamental contribution of the VeriGen framework: it successfully transforms a generative, probabilistic LLM into a reliable engineering tool by embedding systematic verification and optimization at every stage of the software development lifecycle. The VeriGen pipeline introduces additional computational overhead compared to a one shot LLM approach, due to the multiple LLM calls required for the Verifier Optimizer loops and the rule based processing at each stage. Across the four case studies,

we measured total time and token usage for both the raw GPT 3.5 baseline and VeriGen. On average, VeriGen required approximately $3.5\times$ more tokens and $3.5\times$ more time, but improved average accuracy from 46.8% to 88.0%. The summary of results for each case study can be found at GitHub repository [39]. This overhead is a worthwhile investment for applications where correctness is critical.

2) COMPARISON OF RELATED WORK WITH VeriGen

To better understand the contribution of VeriGen, it is important to position it against existing research efforts. Prior works have made notable progress in automating individual stages of the software development lifecycle, such as requirements formalization through controlled natural languages [1], [2], [3], class diagram generation from textual descriptions [4], [5], [7], [9], [14], sequence diagram modeling [17], [18], [19], and code generation from UML models or textual inputs [26], [27], [28], [29], [30]. Some studies have also experimented with verification, though often in a limited, rule-based, or manual refinement manner [14], [17], [18], [19].

However, as the literature shows, these contributions remain isolated focusing on one stage at a time without providing a seamless pipeline that spans from natural language requirements to verified design models and skeletal code generation. Moreover, the lack of systematic verification and optimization mechanisms leaves outputs prone to incompleteness, inconsistency, or LLM hallucinations.

The following Table 10 contrasts related works with VeriGen, highlighting how our proposed tool integrates all stages into a single unified framework while embedding a Verifier–Optimizer loop at every phase to ensure correctness, completeness, and stability of generated artifacts.

VI. DISCUSSION, LIMITATIONS AND FUTURE WORK

A. DISCUSSION

VeriGen presents a significant step toward reliable, end-to-end automation of the software development lifecycle by embedding a systematic Verifier-Optimizer loop within a generative AI pipeline. This architecture directly mitigates the well-known limitations of raw LLMs such as hallucinations, omissions, and inconsistencies transforming them from unreliable oracles into supervised generators within a disciplined engineering process. The framework's validated performance across multiple case studies demonstrates its ability to produce stable, correct, and traceable artifacts from natural language requirements, effectively bridging the historical fragmentation between automated requirements, design, and code generation. VeriGen can be integrated into CI/CD pipelines as a requirements to prototype automation step, enabling rapid validation. Its advantages over recent end to end LLM based approaches include the systematic Verifier Optimizer loop that ensures artifact correctness across the pipeline, a feature absent in most existing tools. For application to large scale, multi module enterprise systems,

TABLE 9. VeriGen end-to-end performance summary.

Metrics	LMS (VeriGen)	LMS (GPT)	MOS (VeriGen)	MOS (GPT)	ZCB (VeriGen)	ZCB (GPT)	DHS (VeriGen)	DHS (GPT)
SNL Accuracy	89.8%	59.6%	79.9%	23.9%	94.1%	29.3%	88.3%	36.9%
CD Accuracy	100%	71.6%	79%	32%	76.3%	55.5%	96.2%	65.9%
SD Accuracy	100%	70.3%	83.7%	33%	79.8%	53.8%	95.6%	69.5%
Code Accuracy	100%	71.6%	79%	32%	76.3%	55.5%	96.2%	65.9%
Overall Evaluation	94.9%	65.5%	79.6%	28%	85.3%	42.3%	92.2%	51.5%
VeriGen End-to-End Accuracy								88%
LLM (ChatGPT) End-to-End Accuracy								46.8%

TABLE 10. Comparison of existing researches with VeriGen.

Researches	Require- ments → SNL	Class Diagram	Sequence Diagram	Code Genera- tion	Verification & Opti- mization	End- to-End Cover- age
[1]–[3]	✓ (CNL / SNL)	×	×	×	×	×
[4], [5], [7], [9], [14]	×	✓	×	×	Limited (manual refinement)	×
[17]–[19]	×	×	✓	×	Limited (rule-based verification)	×
[26]–[30]	×	✓ (input model only)	×	✓	×	×
[15], [16]	×	✓ (SysML/ UML)	×	×	Partial (interactive refinement)	×
VeriGen (Proposed Tool)	✓ (RUPPs SNL via LLM)	✓ (Verified & Opti- mized)	✓ (Verified & Opti- mized)	✓ (Skeletal Java Struc- ture)	✓ (Verifier + Optimizer loop at each stage)	✓ Full End- to-End

Legend: ✓ = Supported, × = Not supported.

the VeriGen pipeline can be decomposed into independent verification units per module, with parallel execution of the Verifier Optimizer loops and a hierarchical aggregation mechanism to ensure overall consistency.

B. LIMITATIONS

While VeriGen demonstrates promising results, the current framework has several limitations that warrant acknowledgment. First, the pipeline's effectiveness remains intrinsically tied to the quality and structure of the input natural language requirements. Although the RUPPs template adds robustness, highly ambiguous, contradictory, or exceptionally terse specifications can still lead to suboptimal outputs or necessitate manual pre-processing. Second, the verification and optimization rules, while comprehensive, are primarily heuristic and syntactic semantic. Consequently, they may fail to capture deeper domain specific logical constraints or business rules that fall outside the predefined patterns, leaving nuanced semantic errors unaddressed.

The heuristic rules, while comprehensive for the evaluated case studies, may not scale to highly complex or novel requirement structures. Future work will investigate learning

based mechanisms to automatically adapt or generate rules from domain specific examples. Furthermore, the current rule set is manually defined and may require significant effort to maintain or adapt when applied to new domains or as software systems evolve over time. Domain specific customization currently relies on manual intervention. The heuristic rule set was derived from common requirement patterns and refined using four case studies, though it may not cover all unusual or domain-specific cases. While the results support VeriGen's claim of repositioning LLMs into disciplined engines, further validation on complex and enterprise-scale systems is needed, and future work will address these limitations by expanding the rule set and case studies. Third, the implementation is optimized for a specific technological stack (e.g., Java, PlantUML) and a standardized SDLC flow. Its generalizability to other programming paradigms (e.g., functional), modeling notations, or lifecycle stages (e.g., generating state diagrams) has not been established, limiting its broader applicability.

The framework was evaluated primarily using GPT-3.5; however, its architecture is model-agnostic. A supplementary evaluation using DeepSeek-V3 on the MOS case study showed that VeriGen improved raw accuracy from 19.9% to 79.6%, comparable to the improvement achieved with GPT-3.5. The summary of results can be found at [39]. This confirms that the Verifier-Optimizer loop generalizes to other LLMs with only minor variations in accuracy. Finally, the evaluation was conducted in a controlled academic setting. The framework's performance, scalability with large scale industrial requirements, and its integration into collaborative development environments require further investigation in real-world contexts to assess its practical viability.

C. FUTURE WORK

Future work will focus on extending the pipeline's scope and adaptability. Key directions include expanding output artifacts to include test cases, API specifications, and deployment configurations, and generalizing code generation to multiple languages and frameworks. Enhancing the optimizer with learning-based mechanisms could allow rule adaptation to specific domains. Complementing this, we envision a configuration interface that allows domain experts to tailor rules

without modifying the core framework, alongside learning based mechanisms for automatic rule synthesis from labeled examples. Furthermore, improving scalability for large scale requirements, integrating with IDEs and CI/CD pipelines, and conducting longitudinal industrial studies will be crucial for transitioning VeriGen from a research prototype to a robust tool for professional software engineering practice. Additionally, we will also investigate architectural extensions for large scale systems, including modular decomposition, distributed verification, and hierarchical consistency management, to enable seamless deployment in complex industrial environments. Another promising direction is to extend the pipeline to automatically generate behavioral code skeletons from verified sequence diagrams, thereby linking dynamic models directly to method implementations. Additionally, we plan to extend the comparative evaluation to all four case studies using DeepSeek V3 and other LLMs to further validate the model agnostic nature of the Verifier Optimizer loop.

VII. CONCLUSION

The pursuit of automated software engineering via Generative AI has been impeded by the inherent unreliability of LLMs prone to hallucinations and inconsistencies and a landscape of fragmented, isolated automation tools. This paper introduces VeriGen, an end-to-end framework designed to overcome these challenges by embedding a continuous Verifier–Optimizer loop throughout the development pipeline. VeriGen automates the transformation from natural language requirements to skeletal code through verified intermediate artifacts. Its core innovation is a feedback mechanism that critically assesses LLM outputs at each stage, categorizing elements as correct, incorrect, missing, or extra, and then heuristically refines them for structural and semantic correctness.

Empirical validation across four diverse case studies demonstrates VeriGen superiority, showing an average end-to-end accuracy of 88%, a substantial improvement over the 46.8% achieved by a raw LLM. The framework nearly eliminated over-specification while achieving high precision and recall, effectively mitigating LLMs' core failure modes.

By providing a trustworthy, automated pathway from requirements to code, VeriGen advances Generative AI in software engineering, repositioning LLMs from black-box oracles into disciplined, suggest-and-refine engines. This work offers a practical blueprint for integrating AI into professional workflows without compromising reliability. Future efforts will expand the pipeline's scope, incorporate adaptive learning, and scale the framework for industrial adoption.

ACKNOWLEDGMENT

For the purpose of language polishing, grammar correction, and improving textual fluency, large language models (LLMs), such as DeepSeek and ChatGPT were utilized. These tools assisted in refining the manuscript's clarity and

readability. All technical content, methodological details, experimental results, interpretations, and conclusion remain the sole intellectual contribution of the research study conducted by the authors.

REFERENCES

- [1] A. Veizaga, M. Alferez, D. Torre, M. Sabetzadeh, and L. Briand, "On systematically building a controlled natural language for functional requirements," *Empirical Softw. Eng.*, vol. 26, no. 4, Jul. 2021, Art. no. 79, doi: [10.1007/s10664-021-09956-6](https://doi.org/10.1007/s10664-021-09956-6).
- [2] J. A. Dávila Quintero, "Controlled natural language models," in *Proc. 2nd Workshop Prolog Educ. (PEG)*, Dallas, TX, USA, 2024.
- [3] P. das Neves Rodrigues and A. R. da Silva, "Controlled natural languages for specifying business intelligence applications," 2023, *arXiv:2311.11895*.
- [4] Z. Babaalla, H. Abdelmalek, A. Jakimi, and M. Oualla, "Extraction of UML class diagrams using deep learning: Comparative study and critical analysis," *Proc. Comput. Sci.*, vol. 236, pp. 452–459, 2024, doi: [10.1016/j.procs.2024.05.053](https://doi.org/10.1016/j.procs.2024.05.053).
- [5] S. Yang and H. Sahraoui, "Towards automatically extracting UML class diagrams from natural language specifications," in *Proc. 25th Int. Conf. Model Driven Eng. Lang. Syst.*, Oct. 2022, pp. 396–403, doi: [10.1145/3550356.3561592](https://doi.org/10.1145/3550356.3561592).
- [6] B. Wang, C. Wang, P. Liang, B. Li, and C. Zeng, "How LLMs aid in UML modeling: An exploratory study with novice analysts," in *Proc. IEEE Int. Conf. Softw. Services Eng. (SSE)*, Jul. 2024, pp. 249–257, doi: [10.1109/SSE62657.2024.00046](https://doi.org/10.1109/SSE62657.2024.00046).
- [7] E. A. Abdelnabi, A. M. Maatuk, and M. Hagal, "Generating UML class diagram from natural language requirements: A survey," in *Proc. IEEE MI-STA*, May 2021, pp. 288–293, doi: [10.1109/MI-STA52233.2021.9464433](https://doi.org/10.1109/MI-STA52233.2021.9464433).
- [8] D. Rosca and L. Domingues, "A systematic comparison of roundtrip software engineering approaches applied to UML class diagram," *Proc. Comput. Sci.*, vol. 181, pp. 861–868, 2021, doi: [10.1016/j.procs.2021.01.240](https://doi.org/10.1016/j.procs.2021.01.240).
- [9] E. A. Abdelnabi, A. M. Maatuk, T. M. Abdelaziz, and S. M. Elakeili, "Generating UML class diagram using NLP techniques and heuristic rules," in *Proc. 20th Int. Conf. Sci. Techn. Autom. Control Comput. Eng. (STA)*, Dec. 2020, pp. 277–282, doi: [10.1109/sta50679.2020.9329301](https://doi.org/10.1109/sta50679.2020.9329301).
- [10] S. Nasiri, Y. Rhazali, M. Lahmer, and N. Chenfour, "Towards a generation of class diagram from user stories in agile methods," *Proc. Comput. Sci.*, vol. 170, pp. 831–837, 2020, doi: [10.1016/j.procs.2020.03.148](https://doi.org/10.1016/j.procs.2020.03.148).
- [11] S. Nasiri and M. Lahmer, "A smart AI framework for backlog refinement and UML diagram generation," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 4, pp. 340–347, 2024, doi: [10.14569/ijacsa.2024.0150474](https://doi.org/10.14569/ijacsa.2024.0150474).
- [12] D. Rouabhia and I. Hadjadj, "Enhancing class diagram dynamics: A natural language approach with ChatGPT," 2024, *arXiv:2406.11002*.
- [13] Y. Meng and A. Ban, "Automated UML class diagram generation from textual requirements using NLP techniques," *Int. J. Informat. Visualizat.*, vol. 8, nos. 2–3, pp. 123–129, Nov. 2024, doi: [10.62527/joiv.8.3-2.3482](https://doi.org/10.62527/joiv.8.3-2.3482).
- [14] M. Ramzan, G. S. Sadiqi, M. S. Bashir, S. Raza, and A. Batool, "A rule-based approach for automatic generation of class diagrams from functional requirements," *J. Comput. Biomed. Informat.*, vol. 7, no. 2, pp. 207–218, 2024.
- [15] S. Zhong, A. Scarinci, and A. Cicirello, "Automatic generation of SysML diagrams using NLP," *Knowledge-Based Syst.*, vol. 257, Jan. 2023, Art. no. 110071, doi: [10.1016/j.knosys.2022.110071](https://doi.org/10.1016/j.knosys.2022.110071).
- [16] A. Akundi, J. Ontiveros, and S. Luna, "Text-to-model transformation: Natural language-based model generation framework," *Systems*, vol. 12, no. 9, p. 369, Sep. 2024, doi: [10.3390/systems12090369](https://doi.org/10.3390/systems12090369).
- [17] M. Jahan, M. M. Hassan, R. Golpayegani, G. Ranjbaran, C. K. Roy, B. Roy, and K. Schneider, "Automated derivation of UML sequence diagrams from user stories: Unleashing the power of generative AI vs. a rule-based approach," in *Proc. ACM Workshop Adv. Model-Driven Softw. Eng.*, 2024, pp. 1–10, doi: [10.1145/3640310.3674081](https://doi.org/10.1145/3640310.3674081).
- [18] A. I. Nur, F. Fasyadin, and Febriansyah, "User scenario 2 Seq: Automated system to generate sequence diagrams from user stories scenario," *AIP Conf. Proc.*, vol. 2554, Jan. 2023, Art. no. 020059, doi: [10.1063/5.0103696](https://doi.org/10.1063/5.0103696).
- [19] A. Mashkooor and A. Egyed, "Evaluating the alignment of sequence diagrams with system behavior," *Proc. Comput. Sci.*, vol. 192, pp. 519–526, 2021, doi: [10.1016/j.procs.2021.01.267](https://doi.org/10.1016/j.procs.2021.01.267).

- [20] A. Alyami, S. F. Pileggi, O. Sohaib, and I. Hawryszkiewicz, "Seamless transformation from use case to sequence diagrams," *PeerJ Comput. Sci.*, vol. 9, p. e1444, Jun. 2023, doi: [10.7717/peerj-cs.1444](https://doi.org/10.7717/peerj-cs.1444).
- [21] E. M. Bouziane, C. Baidada, and A. Jakimi, "Recovering UML2 sequence diagrams from execution traces," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 12, pp. 108–114, 2020, doi: [10.14569/ijacsa.2020.0111213](https://doi.org/10.14569/ijacsa.2020.0111213).
- [22] C. Alvin, B. Peterson, and S. Mukhopadhyay, "Static generation of UML sequence diagrams," *Int. J. Softw. Tools Technol. Transf.*, vol. 23, no. 1, pp. 31–53, Feb. 2021, doi: [10.1007/s10009-019-00545-z](https://doi.org/10.1007/s10009-019-00545-z).
- [23] X. Chen, Q. Liu, F. Mallet, Q. Li, S. Cai, and Z. Jin, "Formally verifying consistency of sequence diagrams for safety-critical systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 3, pp. 1224–1236, Mar. 2021, doi: [10.1109/TASE.49443.2020.00037](https://doi.org/10.1109/TASE.49443.2020.00037).
- [24] S. Al-Fedaghi, "UML sequence diagram: An alternative model," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 5, pp. 510–516, 2021, doi: [10.14569/ijacsa.2021.0120576](https://doi.org/10.14569/ijacsa.2021.0120576).
- [25] S. J. Ali, V. Naganathan, and D. Bork, "Establishing traceability between natural language requirements and software artifacts by combining RAG and LLMs," in *Conceptual Modeling—ER 2024*. Cham, Switzerland: Springer, 2025, pp. 295–314, doi: [10.1007/978-3-031-75872-0_16](https://doi.org/10.1007/978-3-031-75872-0_16).
- [26] G. Antal, R. Vozár, and R. Ferenc, "Assessing GPT-4-vision's capabilities in UML-based code generation," in *Proc. ICSE Companion*, 2024, pp. 253–256, doi: [10.1145/3643795.3648391](https://doi.org/10.1145/3643795.3648391).
- [27] A. R. Sadik, S. Brulin, and M. Olhofer, "LLM as a code generator in agile model driven development," 2024, *arXiv:2410.18489*.
- [28] R. P. Nair and M. G. Thushara, "NL2Code: A hybrid NLP and model-driven framework for automated code generation from natural language and UML," in *Proc. IEEE Int. Students' Conf. Electr., Electron. Comput. Sci. (SCEECS)*, Jan. 2025, pp. 1–6, doi: [10.1109/SCEECS64059.2025.10940505](https://doi.org/10.1109/SCEECS64059.2025.10940505).
- [29] H. Mahmood, A. A. Jilani, and A. Rauf, "Code swarm: A code generation tool based on the automatic derivation of transformation rule set," *Int. J. Softw. Eng. Appl.*, vol. 14, no. 6, pp. 01–11, Nov. 2023, doi: [10.5121/ijsea.2023.14601](https://doi.org/10.5121/ijsea.2023.14601).
- [30] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT," *IEEE Trans. Softw. Eng.*, vol. 50, no. 6, pp. 1548–1584, Jun. 2024, doi: [10.1109/TSE.2024.3392499](https://doi.org/10.1109/TSE.2024.3392499).
- [31] S. Tiwari, P. Shah, and M. Khare, "NL2RT: A tool to translate natural language text into requirements templates (RTs)," in *Proc. IEEE 30th Int. Requirements Eng. Conf. (RE)*, Aug. 2022, pp. 262–263.
- [32] *ACM Digital Library*. Accessed: Sep. 2025. [Online]. Available: <https://dl.acm.org>
- [33] *IEEE Xplore Digital Library*. Accessed: Sep. 2025. [Online]. Available: <https://ieeexplore.ieee.org>
- [34] *SpringerLink*. Accessed: Sep. 2025. [Online]. Available: <https://www.springer.com>
- [35] *Elsevier ScienceDirect*. Accessed: Sep. 2025. [Online]. Available: <https://www.elsevier.com>
- [36] L. Hirschman and H. S. Thompson, "Overview of evaluation in speech and natural language processing," in *Survey of the State of the Art in Human Language Technology*. Cambridge, U.K.: Cambridge Univ. Press, 1997, pp. 409–414.
- [37] R. Grishman and B. Sundheim, "Message understanding conference-6: A brief history," in *Proc. 16th Int. Conf. Comput. Linguistics*, 1996, pp. 466–471.
- [38] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed., Upper Saddle River, NJ, USA: Pearson, 2002.
- [39] VeriGen Project Repository. *VeriGen Results and User Manual*. Accessed: Jan. 14, 2026. [Online]. Available: <https://github.com/Mudassar-Adeel/VeriGen-Results-Code-User-Manual>
- [40] D. Kumar, A. R. Addua, M. Lind, S. Brown, and S. Odion, "AI driven hybrid deep learning and swarm intelligence for predictive maintenance of smart manufacturing robots in Industry 4.0," *Electronics*, vol. 15, no. 2, 2026, Art. no. 715.
- [41] S. Dou, H. Jia, S. Wu, H. Zheng, M. Wu, Y. Tao, M. Zhang, M. Chai, J. Fan, Z. Xi, R. Zheng, Y. Wu, M. Wen, T. Gui, Q. Zhang, X. Qiu, and X. Huang, "What is wrong with your code generated by large language models? An extensive study," *Sci. China Inf. Sci.*, vol. 69, no. 1, 2026, Art. no. 112107.
- [42] R. Jonnal, J. Yang, Y. Lee, G. Liang, and Z. Cao, "Measuring and improving the efficiency of Python code generated by LLMs using CoT prompting and fine tuning," *IEEE Access*, vol. 13, pp. 119666–119679, 2025.
- [43] X. Bai, S. Huang, C. Wei, and R. Wang, "Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability," *Expert Syst. Appl.*, vol. 269, Apr. 2026, Art. no. 126357.



MUDASSAR ADEEL AHMED received the B.S. degree from Mirpur University of Science and Technology, Mirpur, and the M.S. degree from the National University of Sciences and Technology (NUST), Islamabad, Pakistan, where he is currently pursuing the Ph.D. degree in software engineering with the College of Electrical and Mechanical Engineering. His research interests include model-driven software engineering, artificial intelligence, and machine learning.



MUHAMMAD WASEEM ANWAR received the Ph.D. degree in software engineering from the Department of Computer and Software Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology, Pakistan, in July 2021. He is currently a Postdoctoral Researcher in software engineering with Mälardalen University, Västerås, Sweden. He also has more than ten years of industry experience in software development and system verification, mainly focused on embedded and industrial control systems. His research interests include improving model-based systems engineering (MBSE) with artificial intelligence to manage the design and verification challenges of large-scale embedded systems.



WASAI HAIDER BUTT is currently a tenured Professor with the Department of Computer and Software Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology (NUST), Pakistan. He is also the Head of the Automated Software Engineering Research Group, College of Electrical and Mechanical Engineering, NUST. His research interests include model-driven software engineering, web development, and requirement engineering.



FAROOQUE AZAM is currently a Key Faculty Member with the Department of Computer and Software Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology (NUST), Pakistan. He has been involved in postgraduate teaching and research, since 2007. He had 138 international journals and conference publications, until April 2020. His research interests include model-driven software engineering, model-driven testing, model-driven embedded applications, model-driven web engineering, software design, and architecture. He is a regular member of Pakistan Engineering Council (PEC) evaluation committees and the Higher Education Commission's Technological Development Funding (HEC-TDF).

...